

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Факультет комп'ютерних
Кафедра програмної інженерії

ЗВІТ
З ПЕРЕДАТЕСТАЦІЙНОЇ ПРАКТИКИ

Місце проходження практики:
ХНУРЕ, кафедра ПІ, м.Харків
у період з «13» квітня по «23» травня 2026 р.

Тема індивідуального завдання:
Програмна система для безконтактного замовлення їжі в ресторані з
використанням QR-технологій

Студент гр. ПЗПІ-22-1

_____ (підпис)

Денис ТОКАР

Керівник практики

_____ (підпис)

проф. Зоя ДУДАР

Роботу захищено з оцінкою _____ «22» травня 2026 р.

Комісія:

_____ (підпис)

проф. Зоя ДУДАР

_____ (підпис)

доц. Олександр ВЕЧУР

_____ (підпис)

ас. Михайло БАЛИЧЕВ

Харків 2026

РЕФЕРАТ

Звіт з передатестаційної практики: 79 с., 13 рис., 1 табл., 8 джерел.

Об'єкт дослідження — процес автоматизації обслуговування клієнтів у закладах громадського харчування малого та середнього формату.

Предмет дослідження — методи та засоби розробки програмної системи для безконтактного замовлення їжі на основі QR-технологій із використанням технологічного стеку ReactJS, Node.js та MongoDB.

Мета роботи — проектування та розробка програмної системи для безконтактного замовлення їжі в ресторані з використанням QR-технологій, що забезпечує повний цикл взаємодії гостя із закладом від сканування QR-коду до завершення оплати без обов'язкового залучення обслуговуючого персоналу.

Методи дослідження — порівняльний аналіз існуючих програмних рішень у сфері автоматизації ресторанного обслуговування; методології формування вимог до програмного забезпечення відповідно до стандарту IEEE Std 830-1998; методи об'єктно-орієнтованого проектування; шаблони архітектурного проектування веб-застосунків (трирівнева архітектура, multi-tenant SaaS, шарова організація серверного коду); методи проектування документно-орієнтованих баз даних; принципи проектування REST API та протоколу WebSocket для передачі даних у режимі реального часу.

У результаті виконання роботи спроектовано та реалізовано клієнт-серверну систему, що складається з односторінкового веб-застосунку для гостей і персоналу та серверної частини з підтримкою REST API та WebSocket-з'єднань. Сформовано повний комплект функціональних та нефункціональних вимог, що містить 15 функціональних модулів, понад 80 функціональних вимог із унікальними ідентифікаторами та понад 70 критеріїв приймання у форматі Given/When/Then. Спроектовано схему документно-орієнтованої бази даних з 23 колекціями. Реалізовано інтеграцію з зовнішніми сервісами: платіжний шлюз LiqPay, Google OAuth 2.0, AWS S3, Resend, Google Translate API. Реалізовано модель монетизації

Freemium із технічним розподілом функціоналу між безкоштовним та преміум-тарифами на рівні бази даних та проміжного програмного забезпечення.

Практична цінність роботи полягає у створенні програмного продукту, придатного для реального впровадження в закладах громадського харчування малого та середнього формату. Модульна побудова системи забезпечує її гнучкість та можливість подальшого розширення функціональності відповідно до потреб конкретного підприємства. Запропонована модель монетизації Freemium робить продукт фінансово доступним для закладів малого формату, що було визначено як ключова ринкова прогалина за результатами аналізу предметної галузі.

Прогнозні припущення щодо розвитку об'єкта дослідження — подальше зростання попиту на безконтактні цифрові рішення у сфері HoReCa та поширення моделі BYOD (Bring Your Own Device) як основного каналу взаємодії гостя із закладом харчування.

Ключові слова: QR-ТЕХНОЛОГІЇ, БЕЗКООНТАКТНЕ ЗАМОВЛЕННЯ, АВТОМАТИЗАЦІЯ РЕСТОРАНУ, SAAS, ОДНОСТОПІНКОВИЙ ВЕБЗАСТОСУНОК, REACTJS, NODE.JS, MONGODB, WEBSOCKET, REST API, BYOD, FREEMIUM, KDS.

ЗМІСТ

2.1 Загальна характеристика предметної галузі	14
2.2 Огляд існуючих програмних рішень	15
2.2.1 Poster POS	15
2.2.2 OddMenu.....	15
2.2.3 Choice QR.....	16
2.2.4 Kavapp QR.....	17
2.3 Порівняльний аналіз існуючих рішень	17
3.1 Концепція системи та об'єкт автоматизації.....	20
3.2 Рольова модель системи	20
3.3 Функціональні модулі системи.....	21
3.4 Позиціонування на ринку	23
3.5 Модель монетизації.....	24
3.6 Обґрунтування вибору технологічного стеку	25
4.1 Огляд функціональних вимог	27
4.1.1 Модуль QR-менеджменту та ідентифікація столика	27
4.1.2 Модуль меню: інгредієнти, модифікатори та добавки.....	28
4.1.3 Модуль формування замовлення: кошик та групи подачі.....	30
4.1.4 Модуль управління замовленням столику	31
4.1.5 Модуль передачі замовлення на кухню (KDS)	33
4.1.6 Модуль виклику офіціанта.....	34
4.1.7 Модуль відстеження статусу замовлення.....	35
4.1.8 Модуль оплати.....	36
4.1.9 Модуль аналітики та звітності.....	37
4.1.10 Модуль генератора PDF-меню	38
4.1.11 Модуль системи відгуків.....	39
4.1.12 Модуль авторизації та облікових записів.....	40
4.1.13 Модуль управління меню.....	41
4.1.14 Модуль управління персоналом та onboarding ресторану	42
4.1.15 Модуль офлайн-режиму клієнтського інтерфейсу (PWA)	44
4.2 Огляд нефункціональних вимог	45
4.2.1 Вимоги до продуктивності.....	45
4.2.2 Вимоги до безпеки	46
4.2.3 Атрибути якості.....	46
4.2.4 Вимоги до сумісності.....	47
4.2.5 Вимоги до логування	47
4.2.6 Вимоги до моніторингу	47
4.3 Use Case діаграми системи за ролями	47
4.3.2 Use Case діаграма ролі «Офіціант»	48
4.3.3 Use Case діаграма ролі «Кухар»	49
4.3.4 Use Case діаграма ролі «Адміністратор»	50
4.4 Вимоги до тарифної моделі.....	52
5.1 Загальна архітектура системи	53

5.2 Архітектура клієнтської частини.....	55
5.2.1 Технологічний стек та інструменти збірки	55
5.2.2 Структурна організація компонентів	55
5.2.3 Управління станом через контексти React	57
5.2.4 Архітектура WebSocket-клієнта	58
5.2.5 Підтримка PWA та офлайн-режиму	58
5.3 Архітектура серверної частини.....	58
5.3.1 Технологічний стек та модульна організація.....	58
5.3.2 REST API та принципи проектування	60
5.3.3 Middleware-конвеєр обробки запитів.....	60
5.3.4 Реалізація розподілу тарифів через middleware	61
5.4 Проектування бази даних	61
5.4.1 Обґрунтування вибору MongoDB	61
5.4.2 Огляд ключових колекцій	61
5.4.3 Стратегія цілісності даних	63
5.4.4 Діаграма стану замовлення	64
5.5 Архітектура взаємодії в реальному часі	65
5.5.1 Модель кімнат та підписок	65
5.5.2 Каталог подій та формат повідомлень	66
5.5.3 Механізм event replay та відновлення з'єднання.....	67
5.6 Інтеграція з зовнішніми сервісами	67
5.6.1 Інтеграція з платіжним шлюзом LiqPay	67
5.6.2 Інтеграція з Google OAuth 2.0.....	68
5.6.3 Інтеграція з AWS S3 для зберігання зображень	68
5.6.4 Інтеграція з Resend для транзакційних email	69
5.6.5 Інтеграція з Google Translate API	69
5.7 Безпека та автентифікація	69
5.7.1 Модель автентифікації	69
5.7.2 Авторизація через RBAC	70
5.7.3 Захист від типових атак.....	70
5.7.4 Зберігання паролів та секретів.....	70
5.8 Сценарії взаємодії компонентів системи	71
5.8.1 Повний цикл обслуговування гостя	71
5.8.2 Обробка помилок платіжного шлюзу	72
5.8.3 Відновлення WebSocket-з'єднання та event replay.....	73
Перспективи подальшого розвитку.....	76

ВСТУП

Розвиток інформаційних технологій та стрімке їх поширення суттєво змінюють підходи до організації обслуговування в закладах харчування. Сучасний споживач очікує швидкого та зручного обслуговування з мінімальним залученням персоналу, що формує попит на цифрові рішення, здатні замінити або доповнити традиційну модель роботи ресторану [1].

Існуюча практика прийому замовлень через офіціанта супроводжується низкою проблем: затримками в години пік, людськими помилками, витратами на друк і оновлення паперових меню, відсутністю інструментів для збору та аналізу даних про поведінку клієнтів. Зазначені чинники негативно впливають як на якість клієнтського досвіду, так і на економічну ефективність закладу [1, 2].

QR-технології є доступним і технологічно зрілим інструментом, що дозволяє усунути більшість із перелічених недоліків без значних інфраструктурних витрат з боку закладу [2]. Розміщення QR-коду на столику надає гостю можливість самостійно переглянути меню, сформувати та відправити замовлення безпосередньо на кухню, викликати офіціанта та здійснити оплату — все це в межах єдиного веб-інтерфейсу без встановлення додаткового програмного забезпечення.

Метою даної кваліфікаційної роботи є проєктування та розробка програмної системи для безконтактного замовлення їжі в ресторані з використанням QR-технологій.

Для досягнення поставленої мети необхідно вирішити такі завдання:

- дослідити предметну галузь та проаналізувати наявні програмні рішення у сфері автоматизації ресторанного обслуговування;
- сформувати функціональні та нефункціональні вимоги до системи;
- спроектувати архітектуру програмної системи та структуру бази даних;
- реалізувати клієнтський інтерфейс у вигляді SPA для гостей ресторану та панелі керування для персоналу;

- розробити серверну частину на основі REST API з підтримкою WebSocket для передачі замовлень у режимі реального часу;

- реалізувати розширені модулі системи: аналітику продажів, генерацію PDF-меню, систему зворотного зв'язку, CRM-елементи та фінансовий модуль з підтримкою онлайн-оплати і розділення рахунку.

Об'єктом дослідження є процес автоматизації обслуговування клієнтів у закладах громадського харчування.

Предметом дослідження є методи та засоби розробки програмної системи на основі QR-технологій з використанням стеку ReactJS, Node.js та MongoDB.

Практична цінність роботи полягає у створенні програмного продукту, придатного для реального впровадження в закладах громадського харчування. Модульна побудова системи забезпечує її гнучкість та можливість подальшого розширення функціональності відповідно до потреб конкретного підприємства.

1 ОПИС ПІДПРИЄМСТВА

Кафедра програмної інженерії готує студентів за трьома ступенями вищої освіти: бакалаврів за спеціальністю 121 «Інженерія програмного забезпечення» (освітня програмою «Програмна інженерія»); магістрів за спеціальністю 121 «Інженерія програмного забезпечення» (за освітньо-науковою програмою «Інженерія програмного забезпечення»); докторів філософії – за спеціальністю 121 «Інженерія програмного забезпечення». Навчальні плани узгоджено з міжнародними стандартами підготовки програмістів.

Форми навчання: очна, заочна та заочно-дистанційна.

Студенти мають широкі можливості вільного вибору дисциплін. Деякі предмети викладаються англійською мовою в окремо сформованих групах.

Починаючи з 3 курсу студенти проходять практику в провідних ІТ-компаніях Харкова та України. Договір про співробітництво між кафедрою ІІ ХНУРЕ та компаніями Global Logic, EPAM Systems, NIX Solutions та Sigma Software дозволив створити на кафедрі лабораторії з розробки сучасного програмного забезпечення. Відкрито академічний бізнес-інкубатор «YEP!» та YEP!-клуб, які на базі ХНУРЕ є майданчиком для підтримки студентських ініціатив та стартапів.

Практикується залучення провідних спеціалістів фірм до викладання професійно-орієнтованих дисциплін, здійснюється цільова підготовка студентів старших курсів у формі безкоштовних факультативних занять від компаній.

Під час навчання студенти підвищують свій професійний рівень, беруть участь у роботі семінарів і гуртків, організованих на базі наукових лабораторій. Так, у гуртку «Програміст» студенти здобувають знання та навички з виконання завдань з програмування на різних олімпіадах, у лабораторії GameDev навчають студентів розробляти ігрові проекти, а в проекті «IT Talents» поглиблено вивчають складні алгоритми з практичним їх застосуванням у різних галузях ІТ.

Неодноразово студенти кафедри ІІ посідали призові місця на українських та міжнародних змаганнях з програмування, гідно представляли кафедру та університет на виставках, конкурсах, конференціях та форумах.

У галузі освіти та науки кафедра ПІ співпрацює з Ліннеус університетом м. Вакхо (Linnaeus University), Швеція, а також з Університетом Економіки (WSG) м. Бидгощ, Польща. Студенти мають можливість отримати два дипломи – український і європейський рівнів бакалавра або магістра, а також проходити стажування та брати участь у літніх і зимових школах з програмування.

Основні наукові дослідження кафедри виконуються за напрямками:

- створення нового покоління обчислювальних систем і технологій;
- педагогіка в нових освітніх середовищах, дистанційне навчання;
- проектування систем штучного інтелекту;
- розробка математичних моделей механізмів людського інтелекту;
- розробка формального апарату методів логіки, алгебри, лінгвістичної алгебри і системи логічної підтримки проектування нових інформаційних технологій;
- сучасні технології інтеграції гетерогенних розподілених джерел даних і парадигма якості програмного забезпечення автоматизованих систем обробки інформації;
- програмні засоби автоматизованого формування інформаційного простору навчального процесу;
- інформаційні технології дистанційного навчання та електронної комерції; побудова, розробка і реалізації білінгових систем;
- інтелектуальний аналіз даних;
- розвиток основ теорії сегментації та ідентифікації геометричних об'єктів у режимі реального часу для прикладних завдань обробки цифрової інформації;
- семантичний аналіз зображень;
- розробка моделей, методів і алгоритмів розпізнавання для біометричних систем;
- знання-орієнтовані технології класифікації, діагностики і прогнозування ситуацій;

- розробка підсистем аналізу зображень для системи обробки й аналізу технічної інформації в галузі медичного прогнозування;
- розробка програмного забезпечення для систем відеоконтролю;
- розробка моделі навчання і програмного середовища для проведення навчання та перевірки знань у довільній предметній галузі;
- розробка систем мобільного навчання.

Кафедра програмної інженерії існує вже 55 років, хоч і не завжди носила цю назву. На кафедрі працюють більше 60 викладачів, у числі яких 8 докторів наук, 14 професорів, 31 доцент, 39 кандидатів наук.

За час свого існування кафедра підготувала більше 500 кандидатів та докторів наук.

Завідує кафедрою Смеляков К.С., член НТР, заступник голови секції 2 НТР, доктор технічних наук, професор.

У 1963 році Харківський гірничий інститут було реорганізовано у Інститут гірничого машинобудування, автоматики та обчислювальної техніки. У цей період створюється кафедра автоматики та телемеханіки, завідувачем якої стає Іванченко Є.Я.

У цьому ж році кафедра автоматики та телемеханіки була перетворена у кафедру промислової електроніки та обчислювальної техніки, завідувачем був назначений доцент Волков А.А. Відокремившись від цієї кафедри, згідно з наказом №62 від 20 вересня 1963 р. ХІГМАОТа створюється кафедра Обчислювальної техніки.

Першим завідувачем кафедри став Рвачов Володимир Логвинович. Разом із ним на кафедру прийшли працювати Шабанов-Кушнарєнко Юрій Петрович, Мурашко Анрі Гаврилович, Шклярів Леонід Йосипович, Путятін Євген Петрович та Тищенко Володимир Володимирович. Кафедра Обчислювальної техніки здійснювала підготовку інженерів зі спеціальності Математичні та лічильно-вирішальні прилади та пристрої. У науковому плані кафедра займалася розробкою

засобів обчислювальної техніки (ОТ), нових методів обчислень, математичним моделюванням властивостей зору людини..

У 1964 г. стався перший розподіл кафедри ОТ. З'являється кафедра Математичного програмування та моделювання та кафедра Математичних машин.

У 1966 г. кафедра Математичного програмування та моделювання розділилася на кафедри Математичного моделювання (ММ) та кафедру Обчислювальної математики. Завідуючим кафедри ММ стає Шабанов-Кушнарєнко Ю.П. У цей час поширюється тематика математичного моделювання психічних функцій людини та приходять на кафедру такі фахівці як Дюбко Геннадій Федорович, Качко Олена Григорівна, Марченко Юрій Сергійович, Ловицький Володимир Адріанович.

Починаючи з 1969 року кафедра Математичного моделювання здійснює підготовку спеціалістів зі спеціальності Прикладна математика, спеціалізація – математичне забезпечення ЕОМ і АСУ.

У 1975 р. наказом №86 від 15 травня 1975 р. ХІРЕ кафедру Математичного моделювання перейменовано у кафедру Програмного забезпечення ЕОМ (ПЗ ЕОМ). Період 1969-1984 рр. можна охарактеризувати як період розвитку нових наукових напрямків на кафедрі: розпізнавання зорових образів, реалізація біонічних моделей засобами обчислювальної техніки, паралельні обчислення. З'являються такі кафедри як кафедра Застосування ЕОМ та кафедра Обчислювальної техніки.

З 1974 по 1984 г. кафедрою завідував Мурашко А.Г.

З 1984 по 1987 г. кафедрою завідував Дюбко Г.Ф.

У 1987 р. знову відбувається реорганізація. Кафедри ПЗ ЕОМ та ОТ об'єднуються під назвою ПЗ ЕОМ, завідувачем кафедри стає Бондаренко Михайло Федорович. У 1994 році Бондаренко М.Ф. був назначений ректором ХІРЕ, і його замісником і вірним помічником стала Дудар Зоя Володимирівна.

У 2001 г. за ініціативою ХНУРЕ створюється Українська асоціація дистанційної освіти, Михайло Федорович стає її президентом. При центрі

післядипломної освіти (директор Дудар Зоя Володимирівна) кафедрою вперше в університеті розпочато підготовку кадрів по дистанційній формі навчання (спеціальність ПЗАС), за ініціативою кафедри (Каук В.І.) у 2002 р. в ХНУРЕ створюється Центр технологій дистанційної освіти.

У 2003 р. знову відбувається розділення кафедри: з'являється кафедра Соціальної інформатики, що влилася у напрямок підготовки Прикладна математика, завідувачою якої стала Соловійова Катерина Олександрівна.

У теперішній час кафедра займається як науковою діяльністю, так і готує бакалаврів за напрямком «Програмна інженерія» (ПІ), магістрів та спеціалістів зі спеціальності «Інженерія програмного забезпечення» (ПЗ) та «Програмне забезпечення систем» (ПЗС). Напрямок «Програмна інженерія» - єдиний напрям у ХНУРЕ, який повністю відповідає європейському стандарту Software Engineering.

З 11 грудня 2011 кафедра ПЗ ЕОМ була перейменована у кафедру Програмної інженерії.

Ось вже понад 15 років студенти кафедри беруть активну участь у командних та особистих першостях та олімпіадах з програмування (тренер – Вечур О.В.), де часто стають переможцями, або займають призові місця: півфінал України (Донецьк, Харків), фінал України (Вінниця), Кубок України з програмування (м.Одеса), півфінал Світу з програмування (Румунія, м. Бухарест), відкрита особиста командна першість Південного Кавказу (Грузія, м. Батумі).

Щороку кафедра Програмної інженерії ХНУРЕ запрошує на навчання велику кількість абітурієнтів, з першого дня занурюючи їх у атмосферу професіоналізму та невпинного самовдосконалення. Щоб відповідати власним високим стандартам, кафедра залучає провідних спеціалістів до викладання професійно-орієнтованих дисциплін, організовує факультативи на базі наукових лабораторій та надає безліч інших можливостей підвищити свій професійний рівень.

Кафедра Програмної інженерії готує програмістів з 1988 року. Впродовж історії розвитку спеціальність мала декілька поступово змінених назв – ПЗОТАС – ПЗАС – ПІ – ПЗ.

Інженерія програмного забезпечення – це інженерна дисципліна, яка пов’язана зі всіма аспектами виробництва програмного забезпечення від початкових стадій створення специфікації до підтримки системи після здачі в експлуатацію.

Індустрія програмування – одна з найбільш перспективних і динамічних галузей. Сучасні ІТ- компанії забезпечують цікаві проекти, професійне зростання та гідну зарплату. Прогнози економічного розвитку галузі в нашій країні стримуються дефіцитом кваліфікованих кадрів.

Під час навчання студенти опановують різні технологічні підходи до інженерії програмного забезпечення, платформи, операційні системи, середовища та мови програмування для систем різного призначення. Опановують навички роботи в команді та навчаються володіти інструментами колективної розробки програмного забезпечення.

Але якнайкраще про якість здобутої освіти говорять професійні успіхи й досягнення її випускників. Вони – успішні та висококваліфіковані спеціалісти у найрізноманітніших галузях комп’ютерних наук, що мають попит на Батьківщині та за кордоном.

2 АНАЛІЗ ПРЕДМЕТНОЇ ГАЛУЗІ

2.1 Загальна характеристика предметної галузі

Ресторанний бізнес належить до галузей із високою операційною інтенсивністю, де швидкість і точність обробки замовлень безпосередньо впливають на задоволеність клієнтів та фінансовий результат підприємства. Традиційна схема обслуговування передбачає послідовний ланцюжок взаємодій: гість — офіціант — кухня — офіціант — гість, кожна ланка якої є потенційним джерелом затримок і помилок [3].

Цифровізація ресторанного сервісу розвивається за кількома напрямками: автоматизація внутрішніх процесів (системи точок продажу — POS-системи, системи управління кухнею — KDS), діджиталізація взаємодії з клієнтом (мобільні застосунки, самообслуговування) та безконтактне обслуговування на основі QR-технологій. Останній напрям набув особливого поширення після 2020 року і на сьогодні є одним із найбільш доступних інструментів цифрової трансформації для малого та середнього бізнесу у сфері готельно-ресторанного бізнесу (HoReCa) [4].

QR-код (Quick Response Code) — двовимірний штрих-код, що кодує текстову інформацію, зокрема URL-адресу. Сканування QR-коду вбудованою камерою смартфона без встановлення додаткових застосунків є ключовою перевагою цієї технології з точки зору доступності для кінцевого користувача. Розміщення індивідуального QR-коду на кожному столику ресторану дозволяє системі автоматично ідентифікувати місце розташування гостя та прив'язати замовлення до конкретного столика. [3]

Технологічну основу сучасних систем безконтактного замовлення складають:

- прогресивні вебзастосунки (PWA) або адаптивні односторінкові застосунки (SPA), що не потребують встановлення;
- програмні інтерфейси застосунків (REST API або GraphQL) для обміну даними між клієнтом і сервером;

- протоколи двостороннього зв'язку (WebSocket) для передачі подій у режимі реального часу;
- хмарні або локальні реляційні та документно-орієнтовані бази даних (БД) для зберігання меню, замовлень і аналітичних даних.

2.2 Огляд існуючих програмних рішень

2.2.1 Poster POS

Poster POS — хмарна система автоматизації ресторану українського походження, що охоплює функції POS-термінала, складського обліку, аналітики та управління персоналом [5]. Система надає можливість створення QR-меню для гостей, однак ця функція реалізована як статичний перегляд позицій без повноцінного циклу замовлення безпосередньо з боку клієнта. Прийом замовлень залишається прерогативою персоналу через планшет або касовий термінал.

З точки зору аналітики Poster POS пропонує розвинений дашборд із показниками виручки, популярності страв та ефективності персоналу. Проте система є комплексним корпоративним продуктом із щомісячною абонентською платою, що робить її фінансово недоступною для малих закладів. Відкритого API для глибокого налаштування клієнтської частини система не надає, що унеможливорює адаптацію інтерфейсу під фірмовий стиль конкретного закладу [5].

2.2.2 OddMenu

OddMenu — спеціалізований SaaS-сервіс для створення цифрових QR-меню ресторанів [6]. На відміну від Poster POS, продукт зосереджений виключно на взаємодії з гостем: сканування QR-коду відкриває адаптивне меню з фотографіями, описами та фільтрацією за категоріями. Інтерфейс оптимізований для мобільних пристроїв і не потребує реєстрації з боку клієнта.

Однак OddMenu є, по суті, інструментом для відображення меню, а не повноцінною системою замовлення [6]. Платформа не підтримує відправлення замовлення на кухню в режимі реального часу, не має модуля онлайн-оплати,

аналітики, системи зворотного зв'язку чи виклику офіціанта. Функціонал розділення рахунку та CRM-елементи також відсутні. Сервіс надає обмежені можливості для брендування інтерфейсу в рамках безкоштовного тарифу.

2.2.3 Choice QR

Choice QR — хмарна платформа, що надає ресторанам інструменти для управління взаємодією з гостями через QR-оплату, замовлення до столика, цифрове меню, бронювання та CRM для гостей [7]. Відмінністю від більшості конкурентів є те, що гості можуть самостійно розмістити або доповнити замовлення у будь-який момент без очікування, а заклад отримує миттєві сповіщення про нові замовлення, які можуть бути одразу передані на кухню.

Платформа також реалізує функцію оплати через QR-код із підтримкою розділення рахунку між гостями: кожен може самостійно обрати позиції для оплати та залишити чайові. При цьому гостям не потрібно реєструватися або встановлювати застосунок. Також є можливість додати функціонал резервації столиків. З точки зору CRM, система збирає клієнтську базу з усіх каналів, підтримує відправлення промоакцій, фільтрацію клієнтів і перегляд зворотного зв'язку, що, за даними самої платформи, забезпечує приріст виручки до 35% [7].

На додаток ця система надає можливість інтеграції з популярними сервісами доставки. А також платформа дає змогу інтегрувати схожі популярні POS системи, зокрема Syrve, Poster, Servio та інші, зі збереженням накопичених даних та автоматичною синхронізацією замовлень.

Проте основним недоліком Choice QR є обмежений функціонал в безкоштовному та найнижчих тарифах, зокрема відкритий API, CRM, пріоритетна підтримка, інтеграція з POS системами та власний домен, доступні лише на вищих тарифних планах [7]. Генерація PDF-меню для фізичного друку в системі відсутня.

2.2.4 Kavapp QR

Kavapp — українська система автоматизації, що охоплює продажі, склад, логістику, персонал і фінанси для закладів харчування та магазинів, синхронізована через три спеціалізовані застосунки: для менеджменту й аналізу, для продажів і фіскалізації, а також для управління логістикою та складом [8].

Серед маркетингових інструментів платформа включає QR-меню, програми лояльності та знижки. Проте QR-меню в Kavapp реалізоване як допоміжний інструмент у межах ширшої POS-системи, а не як самостійний канал взаємодії гостя з кухнею [8]. Самостійне оформлення замовлення клієнтом через QR-інтерфейс без участі персоналу не передбачено. Система орієнтована насамперед на автоматизацію роботи персоналу, а не на безконтактний клієнтський досвід. Функції виклику офіціанта через інтерфейс, розділення рахунку та генерації PDF-меню відсутні.

2.3 Порівняльний аналіз існуючих рішень

У таблиці 2.1 наведено порівняння розглянутих систем за ключовими функціональними та технічними характеристиками.

Таблиця 2.1 — Порівняльний аналіз існуючих рішень

Характеристика	Poster POS	OddMenu	Choice QR	Kavapp QR	Система, що розробляється
QR-меню для гостей	Так	Так	Так	Так	Так
Самостійне замовлення гостем	Ні	Ні	Так	Ні	Так
Передача замовлення на кухню в реальному часі	Через персонал	Ні	Так	Через персонал	Так
Виклик офіціанта через інтерфейс	Ні	Ні	Ні	Ні	Так
Редагування замовлення «на ходу»	Ні	Ні	Так	Ні	Так
Онлайн-оплата	Ні	Ні	Так	Ні	Так

Продовження таблиці 2.1

Характеристика	Poster POS	OddMenu	Choice QR	Kavapp QR	Система, що розробляється
Розділення рахунку (split bill)	Ні	Ні	Так	Ні	Так
Аналітика та звітність	Розширена	Відсутня	Розширена	Базова	Базова
Генерація PDF-меню	Ні	Ні	Ні	Ні	Так
Система зворотного зв'язку	Ні	Ні	Так	Ні	Так
CRM / історія замовлень	Частково	Ні	Так	Частково	Так
Орієнтація на малий бізнес	Частково	Так	Частково	Так	Так

Проведений аналіз дозволяє розподілити розглянуті рішення на три умовні категорії. До першої належать комплексні POS-платформи (Poster POS, Kavapp), які мають розвинений внутрішній функціонал для персоналу, але не забезпечують повноцінного самостійного замовлення гостем через QR-інтерфейс і є функціонально та фінансово надлишковими для закладів малого і середнього розміру. До другої категорії відносяться вузькоспеціалізовані сервіси цифрового меню (OddMenu), що є доступними у впровадженні, проте обмежені виключно функцією відображення інформації. Третю категорію формують повнофункціональні QR-платформи (Choice QR), які найближчі до концепції системи, що розробляється, однак орієнтовані на міжнародний ринок та потребують значних витрат на підключення.

Таким чином, проведений аналіз виявив прогалину на ринку: існуючі рішення або фінансово недоступні для малого бізнесу через надлишковий функціонал (Poster, Choice QR), або обмежуються лише переглядом меню (OddMenu). Система, що розробляється покликана стати збалансованим інструментом, що поєднує автономність гостя (замовлення, оплата) із унікальними локальними запитами, як-от генерація PDF-меню та прямий виклик офіціанта. Це

дозволить закладам малого формату підвищити швидкість і ефективність обслуговування та мінімізувати витрати.

3 ПОСТАНОВКА ЗАДАЧІ

3.1 Концепція системи та об'єкт автоматизації

Об'єктом автоматизації є операційний цикл обслуговування гостя в закладі громадського харчування — від моменту його приходу до завершення оплати. В традиційній моделі цей цикл включає процеси очікування офіціанта, усне або письмове прийняття замовлення, його ручна передача на кухню, відстеження готовності страв персоналом, доставка замовлення та формування рахунку. Кожен із зазначених процесів є потенційним джерелом затримок, людських помилок і додаткового навантаження на персонал.

Система, що розробляється, автоматизує більшість із перелічених процесів, залишаючи людський фактор лише там, де він є необхідним — у безпосередній взаємодії з гостем та приготуванні страв.

В основі системи лежить модель BYOD (Bring Your Own Device), де смартфон гостя виступає персональним терміналом для замовлення без встановлення сторонніх застосунків. Клієнтський інтерфейс реалізований у форматі SPA (Single Page Application), що забезпечує миттєве завантаження, коректну роботу на мобільних пристроях різних платформ та можливість роботи за нестабільного з'єднання.

Водночас система не виключає класичної моделі обслуговування: офіціант може самостійно сформувати або відредагувати замовлення через власний інтерфейс. Така гібридна модель дозволяє закладу адаптувати систему під власний формат роботи без зміни усталених процесів обслуговування.

3.2 Рольова модель системи

Система обслуговує чотири типи користувачів із розмежованим доступом.

Гість є основним зовнішнім користувачем системи. Доступ до меню здійснюється через сканування QR-коду без обов'язкової реєстрації. Гість може переглядати меню з фільтрацією за категоріями, формувати кошик, відстежувати

статус приготування замовлення в режимі реального часу, доповнювати замовлення після його відправлення, викликати офіціанта через інтерфейс та здійснювати онлайн-оплату з можливістю розділення рахунку. Авторизація через Google-акаунт надає доступ до збереженої історії замовлень та системи відгуків.

Кухар працює з системою через інтерфейс KDS (Kitchen Display System) — спеціалізований дисплей на кухні, що отримує нові замовлення миттєво через WebSocket-з'єднання. Кухар керує станами приготування кожної позиції у замовленні.

Офіціант має доступ до мобільної панелі персоналу з інтерактивною картою залу, що відображає поточний стан кожного столика: вільний, зайнятий або очікує на рахунок. Офіціант може редагувати активні замовлення за зверненням гостя та отримує сповіщення про виклик.

Адміністратор має повний доступ до всіх модулів системи: управління персоналом та рівнями доступу, конструктор меню з управлінням цінами та категоріями, модуль аналітики, генератор PDF-меню, налаштування QR-кодів для столиків та налаштування параметрів ресторану.

3.3 Функціональні модулі системи

Модуль QR-менеджменту забезпечує генерацію унікальних QR-кодів для кожного столика з прив'язкою до його фізичного розташування в залі. Для підвищення гнучкості реалізовано динамічний QR з проміжним редиректом, що дозволяє змінювати цільову URL-адресу без перевипуску коду. Після сканування система автоматично ідентифікує столик і прив'язує до нього всі подальші дії гостя в межах сесії, а механізм Smart Session Recovery відновлює перервану сесію при повторному скануванні або перезавантаженні сторінки.

Модуль замовлень у реальному часі реалізує передачу підтверджених замовлень від клієнтського інтерфейсу до кухонного дисплея через WebSocket-з'єднання без перезавантаження сторінки. Система підтримує одне активне замовлення на столик: повторне сканування QR-коду надає гостю доступ до

перегляду меню без можливості розміщення нового замовлення. Статус замовлення оновлюється одночасно на кухонному дисплеї та в інтерфейсі гостя, а функція груп подачі дозволяє гостю об'єднувати страви в блоки для їх одночасного приготування та подачі.

Модуль кухонного дисплея (KDS) функціонує у двох режимах відображення: Order View, що показує замовлення як єдину картку, та Table View, що групує всі активні позиції по столиках. Персонал кухні послідовно переводить страви між статусами, а зміна статусу миттєво відображається в інтерфейсі гостя через WebSocket.

Модуль виклику персоналу підтримує два сценарії взаємодії гостя з офіціантом. Загальний виклик доступний у будь-який момент сесії для вирішення довільних питань. Виклик для готівкової оплати стає доступним лише після того, як усі страви замовлення отримали статус поданих, що унеможливлює передчасне закриття рахунку.

Фінансовий модуль підтримує як сповіщення для персоналу про оплату готівкою, так і повноцінну онлайн-оплату через інтегрований платіжний шлюз LiqPay. Онлайн-оплата, аналогічно до готівкової, стає доступною лише після подачі всіх страв. Операції скасування замовлення (Cancelled), є інструментами офіціанта або адміністратора і недоступні гостю.

Модуль меню забезпечує повноцінне управління структурою пропозиції закладу через CRUD-операції над стравами, категоріями, інгредієнтами та додатками (add-ons). Гість може переглядати детальний склад кожної позиції, а наявність модифікаторів і add-ons дозволяє персоналізувати замовлення без залучення персоналу.

Модуль аналітики та звітності надає адміністратору дашборд із показниками продажів, середнього чека та популярності окремих позицій меню у вигляді графіків і зведених таблиць за обраний період.

Генератор PDF-меню автоматично формує стилізований друкований документ на основі актуальних даних із бази даних, що призначений для фізичного друку та розміщення в залі.

Модуль авторизації реалізує повноцінну систему автентифікації, що підтримує два методи входу: за допомогою електронної пошти та пароля, а також через Google OAuth 2.0. Модуль також охоплює управління персоналом та процедуру створення нового ресторану.

Система відгуків дозволяє авторизованим гостям залишати оцінки окремим стравам та закладу в цілому після завершення замовлення.

3.4 Позиціонування на ринку

Система позиціонується як спеціалізоване SaaS-рішення (Software as a Service) для закладів харчування малого та середнього формату. Модель передбачає надання повного функціоналу платформи як хмарної послуги: заклад не купує програмне забезпечення і не розгортає власну інфраструктуру — натомість отримує доступ до готової системи без залучення технічних спеціалістів для впровадження.

Ключова відмінність від конкурентів полягає у цільовому фокусі: на відміну від комплексних POS-платформ, система не претендує на повну автоматизацію всіх бізнес-процесів закладу, а вирішує конкретну задачу — забезпечити повний цикл взаємодії гостя із закладом від сканування QR-коду до завершення оплати. Порівняно з вузькоспеціалізованими сервісами цифрового меню система надає значно ширший операційний функціонал за зіставну вартість.

Унікальною конкурентною перевагою системи є розширений real-time функціонал, що забезпечує миттєвий двосторонній зв'язок між гостем та персоналом. На відміну від статичних аналогів, система дозволяє клієнту викликати офіціанта одним натисканням у веб-інтерфейсі та здійснювати редагування або доповнення замовлення «на ходу» без залучення персоналу. Додатковою перевагою є гібридна модель обслуговування, що поєднує самостійне

замовлення гостем і класичну роботу офіціанта, дозволяючи закладу адаптувати систему під власні потреби без повної зміни бізнес процесів.

3.5 Модель монетизації

Система базується на моделі Freemium із поділом функціоналу на два тарифні плани, що дозволяє закладам розпочати цифровізацію без початкових витрат і переходити на платний тариф у міру зростання потреб.

Базовий тариф (Free) надається без обмежень у часі та включає мінімально необхідний для роботи закладу функціонал:

- повний цикл прийому та обслуговування замовлень: QR-меню, самостійне замовлення гостем, кухонний дисплей (KDS), панель офіціанта з картою залу;
- виклик офіціанта через інтерфейс, включаючи виклик для готівкової оплати (cash_payment);
- оплата виключно готівкою через підтвердження офіціантом;
- управління меню з обмеженням: до 5 категорій, до 50 страв, до 3 зображень на страву;
- до 3 акаунтів персоналу сумарно (включно з адміністратором);
- базові налаштування ресторану: назва, адреса, тип кухні, контактні дані, мови інтерфейсу;
- система відгуків вимкнена: гості не мають доступу до форми відгуку.

Розширений тариф (Premium) орієнтований на заклади, які потребують повного функціоналу платформи та інструментів для збільшення прибутку:

- зняття обмежень на меню (необмежена кількість категорій, страв та зображень);
- інтеграція платіжного шлюзу LiqPay для безготівкової оплати онлайн через карту або Apple/Google Pay;
- багаторольовий персонал: підтримка ролей waiter, cook та комбінованої ролі waiter_cook, необмежена кількість акаунтів;

- повноцінний аналітичний модуль: виручка, середній чек, конверсія оплат, час приготування, топ страв, статистика відгуків за весь період;
- модуль управління відгуками для адміністратора (перегляд, фільтрація, видалення);
- активація системи залишення відгуків з боку гостей;
- автоматичний генератор PDF-меню для фізичного друку;
- інтеграція з Google Translate для автоматичного перекладу всіх двомовних полів вводу (назв та описів страв, категорій), що знімає необхідність ручного перекладу при веденні меню.

Розподіл функціоналу реалізовано на рівні бази даних (поле plan у документі ресторану) та контролюється проміжним програмним забезпеченням (middleware) на серверній частині, а також умовним рендерингом елементів інтерфейсу на клієнтській частині..

3.6 Обґрунтування вибору технологічного стеку

Вибір технологій для реалізації системи обумовлений вимогами до продуктивності, масштабованості та швидкості розробки.

ReactJS обрано для реалізації клієнтської частини з огляду на його компонентну архітектуру, що дозволяє незалежно розробляти та повторно використовувати елементи інтерфейсу як для мобільної версії гостя, так і для адміністративного дашборду. Віртуальний DOM забезпечує високу швидкість оновлення інтерфейсу при частих змінах стану — що важливо для відображення статусів замовлень у режимі реального часу.

Node.js обрано для серверної частини завдяки його асинхронній неблокуючій архітектурі, що є оптимальною для обробки великої кількості одночасних з'єднань — зокрема WebSocket-сесій від кількох столиків одночасно. Використання JavaScript як єдиної мови для клієнтської та серверної частин спрощує розробку та підтримку кодової бази.

MongoDB обрано як базу даних завдяки документно-орієнтованій моделі зберігання даних, що природно відповідає структурі меню ресторану: вкладені категорії, варіації страв і модифікатори зручно представляються у форматі JSON-документів без необхідності складних реляційних зв'язків. Гнучка схема дозволяє оперативно змінювати структуру меню без міграцій бази даних.

Google API використовується для реалізації автентифікації гостей за схемою OAuth 2.0, що забезпечує максимально швидкий вхід без необхідності повільної реєстрації акаунта та підвищує довіру користувачів завдяки знайомому та швидкому механізму авторизації.

4 ФОРМУВАННЯ ВИМОГ

4.1 Огляд функціональних вимог

Функціональні вимоги системи розподілені на п'ятнадцять логічних модулів, кожен з яких відповідає окремому бізнес-сценарію. Нижче наведено повний опис кожного модуля системи описаного в SRS документі.

4.1.1 Модуль QR-менеджменту та ідентифікація столика

4.1.1.1 Опис та пріоритет

Модуль забезпечує ідентифікацію столика через унікальний QR-код та керування життєвим циклом гостьової сесії. Ключовою особливістю є механізм Smart Session Recovery, що відновлює попередню сесію при повторному скануванні замість створення нової, а також гарантія одного активного замовлення на столик. Пріоритет: критичний.

4.1.1.2 Функціональні вимоги

- **SYS-QR-01:** Short-code формату /qr/{shortCode}; 4–8 символів [A-Z0-9]; криптографічно безпечна генерація (crypto.randomBytes)
- **SYS-QR-02:** Сервер при GET /qr/{shortCode} перевіряє cookies на активний session token для поточного tableId
- **SYS-QR-03:** Прив'язка short-code → tableId зберігається в БД та може змінюватись адміном
- **SYS-QR-04:** Адмін може тимчасово деактивувати столик без зміни фізичного QR
- **SYS-QR-05:** Session token має термін дії 24 години (Max-Age: 86400); після закінчення генерується нова сесія
- **SYS-QR-06:** Session token зберігається в HTTP cookie з атрибутами HttpOnly, Secure, SameSite=Strict
- **SYS-QR-07:** Адмін завантажує QR-зображення у форматі PNG або PDF для одноразового друку
- **SYS-QR-08:** Карта залу в панелі адміна відображає поточний стан кожного столика

- **SYS-QR-09:** Якщо session token активний — сервер повертає існуючу сесію (кошик, статус замовлення) замість створення нової (Smart Session Recovery)
- **SYS-QR-10:** Нова сесія створюється лише якщо: немає токена в cookies / token прострочений / token анульований / token належить іншому tableId
- **SYS-QR-11:** При переведенні столика в статус «Вільний» система інвалідує всі активні session token цього tableId автоматично
- **SYS-QR-12:** При GET /qr/{shortCode} для неіснуючого shortCode сервер повертає HTTP 404 з інформаційною сторінкою
- **SYS-QR-13:** При скануванні QR столика з активним замовленням — відповідь містить прапорець tableHasActiveOrder: true; новий Order не створюється

4.1.1.3 Критерії приймання

- **AC-QR-01:** Given shortCode існує та активний, When гість сканує QR вперше, Then HTTP 302 до /menu з новим sessionToken за ≤ 500 мс.
- **AC-QR-02:** Given гість має активний token для tableId T1, When повторно сканує QR, Then повертається та сама сесія; новий Order не створюється.
- **AC-QR-03:** Given shortCode не існує, When GET /qr/{unknownCode}, Then HTTP 404 «QR-код не знайдено».
- **AC-QR-04:** Given столик деактивований, When гість сканує QR, Then «Столик тимчасово недоступний»; Session та Order не створюються.
- **AC-QR-05:** Given столик → «Вільний», Then усі session token цього tableId анулюються; наступне сканування гарантовано створює нову сесію.
- **AC-QR-06:** Given за столиком вже є активне замовлення, When другий гість сканує QR, Then відповідь містить tableHasActiveOrder: true; гість бачить меню без кнопки «Підтвердити замовлення».

4.1.2 Модуль меню: інгредієнти, модифікатори та додатки

4.1.2.1 Опис та пріоритет

Модуль реалізує відображення меню з підтримкою гнучкої кастомізації страв через знімні інгредієнти, додатки (add-ons) та обов'язкові варіанти вибору

(ComponentGroups). Передбачено клієнтський пошук на основі кешованих даних для миттєвої реакції інтерфейсу. Пріоритет: високий.

4.1.2.2 Функціональні вимоги

- **SYS-MENU-01:** Кожна страва повинна мати список інгредієнтів, що керується адміном
- **SYS-MENU-02:** Адмін позначає кожен інгредієнт як «можна прибрати» або «обов'язковий»
- **SYS-MENU-03:** Гість може прибрати лише інгредієнти, позначені адміном як дозволені для виключення
- **SYS-MENU-04:** Кожна страва може мати перелік add-ons; кожен add-on має назву та ціну (0 або більше)
- **SYS-MENU-05:** Гість може обрати один або кілька add-ons до страви
- **SYS-MENU-06:** Кожна страва може мати перелік ComponentGroups в якій клієнт обов'язково має обрати одну
- **SYS-MENU-07:** Гість може залишити довільний текстовий коментар до кожної окремої позиції (максимум 300 символів), валідація на клієнті та сервері
- **SYS-MENU-08:** Виключені інгредієнти, add-ons, ComponentGroups та коментар передаються на KDS разом із позицією
- **SYS-MENU-09:** Позиції зі стоп-листа відображаються як недоступні при завантаженні та перевіряються перед створенням замовлення
- **SYS-MENU-10:** При недоступності фото відображається placeholder без порушення відображення решти даних страви
- **SYS-MENU-11:** Пошук за назвою страви доступний глобально по всьому меню та локально в межах категорії; фільтрація case-insensitive з частковим співпадінням; реалізується на клієнті без запиту до сервера на основі кешованих даних меню
- **SYS-MENU-12:** Поле пошуку присутнє на головній сторінці меню (глобальний) та на сторінці кожної категорії (локальний)
- **SYS-MENU-13:** Результати пошуку оновлюються при кожному введеному символі

4.1.2.3 Критерії приймання

- **AC-MENU-01:** Given страва має 5 інгредієнтів (3 знімних, 2 обов'язкових), Then знімні мають чекбокс, обов'язкові — ні.
- **AC-MENU-02:** Given страва має ComponentGroup «Гарнір», Then кнопка «Додати до кошика» неактивна до вибору варіанту.

- **AC-MENU-03:** Given гість виключив «цибулю» та підтвердив замовлення, Then на KDS відображається «Без цибулі».
- **AC-MENU-04:** Given add-on 20 грн, When гість обирає, Then ціна позиції збільшується на 20 грн в реальному часі.
- **AC-MENU-05:** Given гість вводить 301+ символ у коментар, Then символи понад 300 не вводяться; лічильник показує «300/300».
- **AC-MENU-06:** Given кухар додав страву до стоп-листа, Then при спробі замовленні позиція недоступна.
- **AC-MENU-07:** Given меню завантажено, When гість вводить «піца» у глобальний пошук, Then відображаються всі страви з «піца» у назві (case-insensitive) без запиту до сервера.
- **AC-MENU-08:** Given гість перебуває в категорії «Піца» та вводить «маргарит» у локальний пошук, Then відображаються лише страви цієї категорії, що містять «маргарит» у назві.
- **AC-MENU-09:** Given пошуковий запит не знайшов жодної страви, Then відображається «Страв не знайдено».

4.1.3 Модуль формування замовлення: кошик та групи подачі

4.1.3.1 Опис та пріоритет

Гість формує кошик зі стравами та може об'єднати їх у групи подачі — набори страв для одночасного приготування та подачі. На кухонному дисплеї кожна група відображається як монолітний блок зі спільним статусом, що дозволяє синхронізувати подачу складних замовлень.

4.1.3.2 Функціональні вимоги

- **SYS-CART-01:** Гість може об'єднувати страви в іменовані групи подачі в межах одного замовлення
- **SYS-CART-02:** Страви без явного групування потрапляють до «Основної групи» за замовчуванням
- **SYS-CART-03:** Кожна новостворена група автоматично має назву «Група подачі ...» з відповідним номером

- **SYS-CART-04:** На KDS страви — монолітні блоки за групами
- **SYS-CART-05:** Статус групи є спільним для всіх страв у ній; зміна статусу групи змінює статус кожної страви одночасно
- **SYS-CART-06:** Гість отримує WebSocket-сповіщення коли кожна з його груп змінює статус
- **SYS-CART-07:** Гість може змінювати кількість / видаляти позиції до підтвердження
- **SYS-CART-08:** При підтвердженні сервер виконує фінальну валідацію на стоп-лист; за наявності недоступних позицій — HTTP 422 зі списком
- **SYS-CART-09:** Кнопка «Підтвердити замовлення» неактивна для порожнього кошика
- **SYS-CART-10:** При помилці сервера під час підтвердження кошик зберігається на клієнті

4.1.3.3 Критерії приймання

- **AC-CART-01:** Given гість додав 3 страви, When натискає «Підтвердити замовлення», Then замовлення з'являється на KDS за ≤ 300 мс.
- **AC-CART-02:** Given гість створив групи «Стартери» та «Основне», When замовлення підтверджено, Then на KDS відображаються 2 окремі блоки з відповідними назвами.
- **AC-CART-03:** Given одна страва потрапила до стоп-листа, When гість натискає «Підтвердити», Then замовлення не відправляється; відображається список недоступних страв з кнопкою «Видалити недоступні».
- **AC-CART-04:** Given кошик порожній, When відображається сторінка кошика, Then кнопка «Підтвердити замовлення» має атрибут disabled.
- **AC-CART-05:** Given сервер повернув HTTP 500 при підтвердженні, When гість бачить помилку, Then кошик незмінний; повторна спроба можлива.

4.1.4 Модуль управління замовленням столику

4.1.4.1 Опис та пріоритет

За кожним столиком може існувати щонайбільше одне активне замовлення (зі статусом open) одночасно. При спробі другого гостя розмістити замовлення за

тим самим столиком система повертає HTTP 409 (TABLE_OCCUPIED). Другий гість, що сканує QR столика, отримує меню в режимі «тільки перегляд» — він може переглядати страви, але не може додавати позиції до кошика або підтверджувати замовлення. Пріоритет: критичний.

4.1.4.2 Функціональні вимоги

- **SYS-ORDER-01:** Якщо декілька людей сканує QR то лише перший хто зробить замовлення буде мати доступ до нього всі інші зможуть лише переглядати меню
- **SYS-ORDER-02:** Сервер перевіряє наявність активного замовлення за tableId перед створенням нового; якщо є — HTTP 409 TABLE_OCCUPIED та tableHasActiveOrder: true
- **SYS-ORDER-03:** Відповідь QR-ендпоінту для столика з активним замовленням містить tableHasActiveOrder: true
- **SYS-ORDER-04:** Гість, який отримав tableHasActiveOrder: true, бачить меню без можливості розмістити замовлення (режим «тільки перегляд»)
- **SYS-ORDER-05:** Гість бачить лише власне замовлення; доступ до чужого → HTTP 403
- **SYS-ORDER-06:** Столик → free коли замовлення переходить у фінальний статус (completed_cash / completed_pay / cancelled) — автоматично
- **SYS-ORDER-07:** Офіціант може вручну закрити столик незалежно від стану замовлення; залишкове активне замовлення → cancelled з причиною «Столик закрито вручну»
- **SYS-ORDER-08:** При вході авторизованого гостя — система перевіряє наявність активного замовлення та відновлює стан замовлення та restaurantId автоматично

4.1.4.3 Критерії приймання

- **AC-ORDER-01:** Given за столиком є активне замовлення (open), When другий гість сканує QR, Then він бачить меню з неактивним кошиком; кнопка «Підтвердити замовлення» неактивна
- **AC-ORDER-02:** Given замовлення → completed_pay, Then столик → «Вільний» за ≤ 300 мс
- **AC-ORDER-03:** Given двоє ініціюють POST /orders одночасно, Then другий → HTTP 409; лише одне замовлення створено

- **AC-ORDER-04:** Given авторизований гість входить до акаунта і має активне замовлення, Then його замовлення restaurantId відновлюється автоматично; гість перенаправляється до меню
- **AC-ORDER-05:** Given не авторизований гість який має активне замовлення втрачає до нього доступ(очищення cookie або зміна пристрою), Then після звернення до персоналу офіціант може ввімкнути режим відновлення і при повторному скануванні QR сесія з даним про замовлення буде відновлена

4.1.5 Модуль передачі замовлення на кухню (KDS)

4.1.5.1 Опис та пріоритет

Модуль забезпечує миттєву передачу підтверджених замовлень на кухонний дисплей через WebSocket. Статуси страв змінюються однонаправлено за моделлю waiting → cooking → ready → served з короткочасною можливістю відкату. Пріоритет: критичний.

4.1.5.2 Функціональні вимоги

- **SYS-KDS-01:** Нові замовлення на KDS без ручного оновлення (WebSocket ORDER_NEW), затримка ≤ 300 мс
- **SYS-KDS-02:** Картка замовлення містить: номер столика, orderId, групи подачі зі стравами, виключені інгредієнти, add-ons, коментарі та час надходження
- **SYS-KDS-03:** Групи подачі відображаються як монолітні блоки з єдиною кнопкою зміни статусу
- **SYS-KDS-04:** Статуси групи: Очікує → Готується → Готово (однаправлений перехід; зниження заборонено)
- **SYS-KDS-05:** Кухар може додавати/знімати позиції зі стоп-листа
- **SYS-KDS-06:** Зміна стоп-листа відображається в меню всіх клієнтських та блокує замовлення з ними
- **SYS-KDS-07:** KDS підтримує два режими: Order View (кожне замовлення — окрема картка) та Table View (всі замовлення столика — єдиний блок)
- **SYS-KDS-08:** При розриві WebSocket KDS відображає індикатор та автоматично відновлює з'єднання

- **SYS-KDS-09:** Після відновлення WebSocket сервер надсилає event replay пропущених подій від останнього підтвердженого event_id
- **SYS-KDS-10:** Спроба зниження статусу групи може бути виконана лише в першу хвилину після минулої зміни в інших випадках відхиляється сервером (HTTP 400)

4.1.5.3 Критерії приймання

- **AC-KDS-01:** Given замовлення з 2 групами, Then 2 блоки на KDS які мають сталий порядок за ≤ 300 мс
- **AC-KDS-02:** Given WebSocket KDS розірваний і відновлений, Then актуальний стан замовлень без ручного оновлення за ≤ 5 с.
- **AC-KDS-03:** Given кухар позначив групу «ready», Then GROUP_STATUS_UPDATED $\rightarrow$ пристрій гостя за ≤ 300 мс
- **AC-KDS-04:** Given кухар помилково змінює статус групи до «ready», When кухар намагається повернути «cooking», Then успіх статус повертається, клієнт сповіщений про цю помилку
- **AC-KDS-05:** Given група «ready», When кухар намагається $\rightarrow$ «cooking», Then HTTP 400; статус не змінюється

4.1.6 Модуль виклику офіціанта

4.1.6.1 Опис та пріоритет

Система підтримує два типи виклику офіціанта: загальний виклик (call) — гість може викликати офіціанта у будь-який момент після підтвердження замовлення та виклик для готівкової оплати (cash_payment). Офіціант отримує WebSocket-сповіщення за ≤ 300 мс. Непідтверджені виклики старші 3 хвилин — візуально виділяються. Повторний виклик поки є вже активний ігнорується. Пріоритет: високий.

4.1.6.2 Функціональні вимоги

- **SYS-CALL-01:** Кнопка загального виклику доступна в будь-який момент після підтвердження замовлення

- **SYS-CALL-02:** Кнопка виклику для готівкової оплати (cash_payment) є активною протягом усього замовлення та при натисканні закриває зміну замовлення
- **SYS-CALL-03:** WebSocket-сповіщення на панель офіціанта за ≤ 300 мс для обох типів
- **SYS-CALL-04:** Офіціант підтверджує виклик одним натисканням
- **SYS-CALL-05:** Сповіщення містить: номер столика, orderId, тип виклику, час виклику
- **SYS-CALL-06:** Виклики сортуються по їх часу де перший найдовший
- **SYS-CALL-07:** Повторний виклик від того самого гостя протягом 60 секунд ігнорується; гість отримує повідомлення
- **SYS-CALL-08:** Виклик, що надійшов під час відключення WebSocket офіціанта, доставляється після відновлення з'єднання (event replay)

4.1.6.3 Критерії приймання

- **AC-CALL-01:** Given гість натискає «Викликати офіціанта», Then на панелі офіціанта з'являється сповіщення за ≤ 300 мс.
- **AC-CALL-02:** Given гість натиснув виклик, When повторно протягом 60 с, Then новий WaiterCall не створюється.
- **AC-CALL-03:** Given гість натискає «Оплатити готівкою», Then офіціант отримує WAITER_CALL_CASH за ≤ 300 мс
- **AC-CALL-04:** Given офіціант підтверджує cash_payment виклик, Then замовлення $\rightarrow$ completed_cash; столик $\rightarrow$ free; WebSocket PAYMENT_COMPLETED за ≤ 300 мс

4.1.7 Модуль відстеження статусу замовлення

4.1.7.1 Опис та пріоритет

Гість бачить стан кожної страви в реальному часі через WebSocket. Статус замовлення (open / cancelled / completed_cash / completed_pay) не є похідним від статусів страв і відображає фінансово-операційний стан. Редагування заблоковане для страв зі статусом cooking+; після переходу замовлення у фінальний статус (cancelled / completed_*) — будь-які зміни неможливі. Пріоритет: високий.

4.1.7.2 Функціональні вимоги

- **SYS-STAT-01:** Інтерфейс відображає статус по кожній групі подачі: Прийнято / Готується / Готово
- **SYS-STAT-02:** Оновлення статусу через WebSocket без перезавантаження сторінки
- **SYS-STAT-03:** Гість може доповнити замовлення новими позиціями (лише коли Order.status = open)
- **SYS-STAT-04:** Гість НЕ може видаляти або зменшувати кількість позицій у групі зі статусом «Готується» або «Готово»
- **SYS-STAT-05:** Редагування (видалення/зменшення кількості) дозволене лише для позицій у групах зі статусом «Очікує»
- **SYS-STAT-06:** Після cancelled або completed_* — будь-які зміни → HTTP 409
- **SYS-STAT-07:** При розриві WebSocket клієнт відображає індикатор втрати з'єднання і відновлює його автоматично (≤ 5 с)

4.1.7.3 Критерії приймання

- **AC-STAT-01:** Given кухар змінив статус групи, When подія надходить на пристрій гостя, Then статус оновлюється без перезавантаження за ≤ 300 мс.
- **AC-STAT-02:** Given група має статус «Готується», When гість намагається видалити позицію, Then відображається «Страву вже готують — зверніться до офіціанта»; позиція залишається.
- **AC-STAT-03:** Given Order.status = completed_epay, When спроба зміни, Then HTTP 409

4.1.8 Модуль оплати

4.1.8.1 Опис та пріоритет

Система використовує модель пост-пей. Платіжний шлюз: LiqPay. Гість оплачує замовлення через LiqPay або готівкою. Оплата може бути здійсненна і до закінчення замовлення, але в такому разі подальше редагування замовлення заблоковане. Пріоритет: критичний.

4.1.8.2 Функціональні вимоги

- **SYS-PAY-01:** Кнопка онлайн-оплати (LiqPay) може бути натиснута при стані замовлення open незалежно від стану страв в ньому
- **SYS-PAY-02:** Карткові дані не зберігаються на серверах системи — обробляються виключно LiqPay
- **SYS-PAY-03:** Після закриття замовлення (completed_cash / completed_eraу / cancelled) → столик «Вільний»
- **SYS-PAY-04:** Всі транзакції (CASH, Cancelled, walk-out) зберігаються в БД та незмінному audit log
- **SYS-PAY-05:** Готівкова оплата підтверджується офіціантом через WaiterCall типу cash_payment або вручну офіціантом; після підтвердження Order.status → completed_cash
- **SYS-PAY-06:** Після completed_* або cancelled — будь-які зміни замовлення → HTTP 409
- **SYS-PAY-07:** При LiqPay timeout Order.status без змін; інформативне повідомлення користувачу
- **SYS-PAY-08:** Fallback: повторні запити статусу до LiqPay через 1/5/15 хв при ненадходженні webhook

4.1.8.3 Критерії приймання

- **AC-PAY-01:** Given LiqPay підтверджує, When webhook надходить, Then Order.status → completed_eraу і підтвердження для гостя за ≤ 3 с.
- **AC-PAY-02:** Given LiqPay не відповідає > 30 с, Then повідомлення про помилку; Order.status = open.

4.1.9 Модуль аналітики та звітності

4.1.9.1 Опис та пріоритет

Адміністратор має доступ до аналітичного дашборду для перегляду бізнес-метрик. Дані формуються з транзакцій MongoDB у реальному часі. Пріоритет: середній (розширений тариф).

4.1.9.2 Функціональні вимоги

- **SYS-ANLT-01:** Загальна виручка за обраний період
- **SYS-ANLT-02:** Середній чек замовлення за обраний період
- **SYS-ANLT-03:** Топ-10 найпопулярніших страв за кількістю замовлень

- **SYS-ANLT-04:** Вибір довільного діапазону дат (максимум 365 днів за один запит)
- **SYS-ANLT-05:** Дані формуються з транзакцій MongoDB
- **SYS-ANLT-06:** Кількість та відсоток cancelled випадків за обраний період
- **SYS-ANLT-07:** Середній час приготування замовлення (від cooking до ready) по стравах та категоріях
- **SYS-ANLT-08:** Конверсія оплат: відсоток PAID проти cancelled
- **SYS-ANLT-09:** Для діапазонів > 365 днів — пропозиція вивантаження CSV

4.1.9.3 Критерії приймання

- **AC-ANLT-01:** Given адмін обирає діапазон до 90 днів, When запит виконується, Then дашборд завантажується за ≤ 3 с.
- **AC-ANLT-02:** Given транзакція з типом “cancelled”, When вона включена в аналітику, Then вона входить до статистики cancelled але НЕ до загальної виручки

4.1.10 Модуль генератора PDF-меню

4.1.10.1 Опис та пріоритет

Адміністратор формує PDF-документ із поточним меню для фізичного друку.

Пріоритет: середній(розширений тариф).

4.1.10.2 Функціональні вимоги

- **SYS-PDF-01:** PDF формується на основі поточних даних меню з MongoDB
- **SYS-PDF-02:** PDF містить: назву закладу, категорії, назви страв, описи, інгредієнти та ціни
- **SYS-PDF-03:** Платний тариф — вибір між шаблонами оформлення
- **SYS-PDF-04:** Формат A4, придатний для якісного друку
- **SYS-PDF-05:** При помилці генерації адмін отримує повідомлення «Не вдалось згенерувати PDF. Спробуйте пізніше»

4.1.10.3 Критерії приймання

- **AC-PDF-01:** Given меню містить 100 позицій, When адмін ініціює генерацію, Then PDF файл формується за ≤ 5 с і доступний для завантаження.
- **AC-PDF-02:** Given PDF-рушій недоступний, When адмін ініціює генерацію, Then відображається помилка; інші функції системи залишаються доступними.

4.1.11 Модуль системи відгуків

4.1.11.1 Опис та пріоритет

Авторизовані гості залишають відгуки: про ресторан (один на замовлення) та по одному на кожну замовлену страву. Доступно лише після `Order.status = completed_cash` або `completed_epay`. Пріоритет: середній (розширений тариф).

4.1.11.2 Функціональні вимоги

- **SYS-REV-01:** Відгуки доступні лише авторизованим гостям для замовлень зі статусом `completed_cash` або `completed_epay`
- **SYS-REV-02:** `RestaurantReview` — один на замовлення від одного акаунта
- **SYS-REV-03:** `DishReview` — один на кожну `OrderItem` від одного акаунта
- **SYS-REV-04:** Обидва типи: оцінка 1–5 та необов'язковий коментар (≤ 500 символів)
- **SYS-REV-05:** Адмін може переглядати та видаляти відгуки з фільтрацією за датою, стравою, рейтингом

4.1.11.3 Критерії приймання

- **AC-REV-01:** Given авторизований гість з `completed_epay` замовленням, When залишає `RestaurantReview` з оцінкою 4, Then відгук у панелі адміна.
- **AC-REV-02:** Given вже є `RestaurantReview`, When повторна спроба, Then HTTP 409.
- **AC-REV-03:** Given анонімний гість, When відкриває форму відгуку, Then HTTP 401.

- **AC-REV-04:** Given замовлення не completed_*, Then HTTP 403 «Відгуки доступні лише для завершених замовлень».

4.1.12 Модуль авторизації та облікових записів

4.1.12.1 Опис та пріоритет

Модуль підтримує два паралельні методи входу: email+пароль із bcrypt-хешуванням та Google OAuth 2.0. Для гостей автентифікація опціональна — система підтримує анонімне користування з прив'язкою сесії до QR-сканування. Пріоритет: критичний.

4.1.12.2 Функціональні вимоги

- **SYS-AUTH-01:** Система підтримує реєстрацію та вхід через email+пароль для всіх типів акаунтів
- **SYS-AUTH-02:** Система підтримує вхід через Google OAuth 2.0 для гостей та персоналу
- **SYS-AUTH-03:** Для персоналу (кухар, офіціант, адмін) наявність email+пароля є обов'язковою умовою
- **SYS-AUTH-04:** Гість може замовляти анонімно без будь-якої авторизації
- **SYS-AUTH-05:** При вході нового користувача через Google — система автоматично створює акаунт; ім'я з Google-профілю
- **SYS-AUTH-06:** При повторному вході через Google — система знаходить існуючий акаунт за Google ID
- **SYS-AUTH-07:** Ім'я в профілі можна змінити незалежно від методу створення акаунта
- **SYS-AUTH-08:** Гість і персонал можуть прив'язати Google-акаунт до існуючого email-акаунта
- **SYS-AUTH-09:** Відв'язати Google можна лише за наявності альтернативного методу входу
- **SYS-AUTH-10:** Паролі зберігаються виключно у вигляді bcrypt-хешів (salt-rounds ≥ 12)
- **SYS-AUTH-11:** JWT access-токен діє ≤ 1 годину; refresh-токен зберігається зашифровано
- **SYS-AUTH-12:** API перевіряє роль користувача (RBAC) для кожного захищеного ендпоінта
- **SYS-AUTH-13:** При недоступності Google OAuth система продовжує роботу через email+пароль

- **SYS-AUTH-14:** При вході авторизованого гостя — система перевіряє наявність активного замовлення та відновлює restaurantId автоматично
- **SYS-AUTH-15:** Прямая реєстрація (email+пароль) активується одразу без email-верифікації; для onboarding нового ресторану обов'язкова email-верифікація (/onboarding/check-email → /onboarding/confirm)
- **SYS-AUTH-16:** Кожен гість може за бажанням видалити свій акаунт

4.1.12.3 Критерії приймання

- **AC-AUTH-01:** Given нова реєстрація, Then пароль у БД — bcrypt-хеш; відкритий текст відсутній; акаунт одразу активний.
- **AC-AUTH-02:** Given Google OAuth недоступний, When гість натискає «Увійти через Google», Then відображається помилка; кнопка email-входу залишається функціональною.
- **AC-AUTH-03:** Given персонал роллю «cook», When GET /api/admin/analytics, Then HTTP 403.
- **AC-AUTH-04:** Given гість увійшов виключно через Google (без пароля), When намагається відв'язати Google, Then HTTP 400 «Встановіть пароль перед відв'язуванням Google».

4.1.13 Модуль управління меню

4.1.13.1 Опис та пріоритет

Модуль надає адміністратору повний CRUD-функціонал для всіх сутностей меню: категорій, страв, інгредієнтів, додатків та груп компонентів. Пріоритет: критичний.

4.1.13.2 Функціональні вимоги

- **SYS-MGMT-01:** Адмін може створювати, редагувати та видаляти (soft-delete) категорії
- **SYS-MGMT-02:** Адмін може створювати, редагувати та видаляти (soft-delete) страви
- **SYS-MGMT-03:** unitPrice вже розміщених OrderItem залишається незмінним (snapshot ціни)
- **SYS-MGMT-04:** Адмін керує інгредієнтами страви (знімні / обов'язкові)

- **SYS-MGMT-05:** Адмін керує add-ons страви (назва, ціна)
- **SYS-MGMT-06:** Адмін керує ComponentGroups (обов'язковий вибір варіанту)
- **SYS-MGMT-07:** Адмін керує sortOrder категорій та страв
- **SYS-MGMT-08:** Будь-яка зміна меню → WebSocket MENU_UPDATED всім активним клієнтам
- **SYS-MGMT-09:** Видалення категорії зі стравами → HTTP 409
- **SYS-MGMT-10:** Зображення страв: JPG/PNG/WebP, ≤ 5 МБ

4.1.13.3 Критерії приймання

- **AC-MGMT-01:** Given адмін додає страву за 150 грн, When гість відкриває меню, Then страва відображається за ≤ 500 мс після збереження.
- **AC-MGMT-02:** Given адмін змінює ціну з 100 на 120 грн, When гість вже має її в підтвердженому замовленні, Then unitPrice в OrderItem залишається 100 грн.
- **AC-MGMT-03:** Given адмін видаляє страву (soft-delete), Then MENU_UPDATED → клієнти; страва → «Недоступно».
- **AC-MGMT-04:** Given адмін видаляє категорію зі стравами, Then HTTP 409; категорія не видаляється.

4.1.14 Модуль управління персоналом та onboarding ресторану

4.1.14.1 Опис та пріоритет

Адміністратор керує обліковими записами персоналу. Перший адміністратор реєструється через процедуру Onboarding при створенні закладу в системі. Один обліковий запис адміністратора прив'язаний рівно до одного ресторану. Email-повідомлення для запрошення персоналу не надсилаються — тимчасові паролі відображаються адміністратору в інтерфейсі одноразово. Для onboarding нового ресторану надсилається email-підтвердження. Пріоритет: високий.

Процедура реєстрації нового ресторану в системі:

- Крок 1: Власник / менеджер відкриває сторінку реєстрації та вводить: email, пароль, ім'я, назву та адресу ресторану

- Крок 2: Система створює Restaurant (restaurantId) та обліковий запис першого адміністратора з роллю administrator
- Крок 3: На email надсилається підтверджувальний лист; акаунт активується після підтвердження email
- Крок 4: Адмін входить до системи та може розпочати налаштування: додавати столики, QR-коди, меню та персонал
- Крок 5: Система генерує перший short-code та QR для тестового столика (опційно, «Quick Start»)

4.1.14.2 Функціональні вимоги

- **SYS-STAFF-01:** Перший адміністратор реєструється через Onboarding при створенні ресторану; акаунт активується після email-верифікації (маршрут /onboarding/check-email → /onboarding/confirm)
- **SYS-STAFF-02:** Адмін може створювати акаунти для кухарів та офіціантів (email + тимчасовий пароль)
- **SYS-STAFF-03:** При створенні акаунта система надсилає тимчасовий пароль співробітнику на вказаний email
- **SYS-STAFF-04:** Адмін може змінювати роль: cook ↔ waiter ↔ administrator
- **SYS-STAFF-05:** При зміні ролі всі активні JWT токени анулюються
- **SYS-STAFF-06:** Адмін може деактивувати акаунт; всі активні сесії анулюються
- **SYS-STAFF-07:** Ресторан повинен мати мінімум одного активного адміністратора
- **SYS-STAFF-08:** Адмін не може видалити власний акаунт
- **SYS-STAFF-09:** Список персоналу з фільтрацією за роллю та статусом;

- **SYS-STAFF-10:** Один адміністраторський акаунт прив'язаний рівно до одного ресторану; спроба реєстрації другого ресторану з тим самим email → HTTP 409
- **SYS-STAFF-11:** Адмін може скинути пароль будь-якого співробітника

4.1.14.5 Критерії приймання

- **AC-STAFF-01:** Given адмін створює акаунт кухаря, Then система надсилає тимчасовий пароль на вказаний email; співробітник входить в акаунт з отриманим паролем
- **AC-STAFF-02:** Given адмін деактивує офіціанта, When офіціант намагається увійти, Then HTTP 401 «Акаунт деактивовано»
- **AC-STAFF-03:** Given єдиний адміністратор, When деактивація, Then HTTP 409
- **AC-STAFF-04:** Given власник реєструє ресторан через Onboarding, Then restaurantId та акаунт адміна створені; обліковий запис одразу активний
- **AC-STAFF-05:** Given адмін намагається зареєструвати другий ресторан зі своїм email, Then HTTP 409 «Обліковий запис вже прив'язаний до ресторану»

4.1.15 Модуль офлайн-режиму клієнтського інтерфейсу (PWA)

4.1.15.1 Опис та пріоритет

Клієнтський інтерфейс реалізується як Progressive Web App (PWA) з підтримкою часткового офлайн-режиму через Service Worker. Весь функціонал, що не вимагає серверної взаємодії, повинен залишатися доступним при відсутності інтернет-з'єднання. Session token зберігається в HttpOnly cookie; кошик та черга замовлень — у localStorage. Пріоритет: середній.

4.1.15.2 Функціональні вимоги

- **SYS-PWA-01:** Service Worker кешує статичні ресурси (JS, CSS, шрифти, іконки) при першому завантаженні застосунку

- **SYS-PWA-02:** Дані меню (категорії, страви, інгредієнти, add-ons, ComponentGroups) кешуються після першого успішного завантаження
- **SYS-PWA-03:** Кошик зберігається в localStorage; відновлюється при повторному відкритті навіть без мережі
- **SYS-PWA-04:** Якщо гість натиснув «Підтвердити» в офлайн — замовлення ставиться в чергу в localStorage та відправляється при відновленні з'єднання з нотифікацією «Замовлення відправляється...»
- **SYS-PWA-05:** Черга очікуючих запитів зберігається в localStorage; при закритті та повторному відкритті браузера черга не втрачається
- **SYS-PWA-06:** Клієнт відображає banner «Немає з'єднання» при відсутності мережі; banner зникає при відновленні
- **SYS-PWA-07:** Пошук у меню функціонує офлайн на основі кешованих даних

4.1.15.3 Критерії приймання

- **AC-PWA-01:** Given меню завантажено, When гість вимикає мережу, Then меню, деталі страв, пошук та кошик залишаються доступними
- **AC-PWA-02:** Given гість додав страви до кошика офлайн, When мережа відновлена, Then кошик збережений; гість може підтвердити замовлення
- **AC-PWA-03:** Given гість натиснув «Підтвердити» офлайн, Then система показує «Замовлення відправляється...»; після відновлення мережі замовлення відправляється автоматично; гість отримує підтвердження
- **AC-PWA-04:** Given мережа відсутня, Then відображається banner «Немає з'єднання»; при відновленні banner зникає; статус замовлення оновлюється автоматично за ≤ 5 с

4.2 Огляд нефункціональних вимог

Нефункціональні вимоги визначають якісні характеристики системи та поділяються на сім категорій.

4.2.1 Вимоги до продуктивності

Час завантаження клієнтського інтерфейсу не повинен перевищувати 2 секунди при підключенні 4G. Затримка передачі замовлення через WebSocket — не

більше 300 мс. Час відповіді REST API — не більше 500 мс для 95% запитів. Система розрахована на одночасну обробку до 100 активних WebSocket-з'єднань на один ресторан. Доступність системи (uptime) — не менше 99,5% за календарний місяць.

4.2.2 Вимоги до безпеки

Усі мережеві з'єднання здійснюються виключно через HTTPS та WSS. Автентифікація персоналу реалізована через JWT-токени з обмеженим терміном дії. Паролі зберігаються виключно у вигляді bcrypt-хешів з параметром salt-rounds ≥ 12 . Карткові дані обробляються виключно платіжним шлюзом LiqPay і не потрапляють на сервери системи. Усі API-ендпоінти захищені перевіркою ролі користувача за моделлю RBAC. Реалізовано захист від типових векторів атак: XSS, CSRF, NoSQL-ін'єкцій. Фінансові операції фіксуються в незмінному audit log.

4.2.3 Атрибути якості

Зручність (Usability): гість виконує перше замовлення без інструкцій за не більше ніж три дії. Надійність (Reliability): автоматичне відновлення WebSocket-з'єднання після розриву за не більше 5 секунд. Масштабованість (Scalability): архітектура передбачає горизонтальне масштабування для обслуговування багатьох ресторанів. Супроводжуваність (Maintainability): компонентна структура клієнтської частини та модульна архітектура серверної частини забезпечують незалежне оновлення модулів. Переносимість (Portability): коректне відображення на пристроях з шириною екрана від 320 px до 1920 px. Відновлюваність (Recoverability): Smart Session Recovery дозволяє відновити сесію після закриття браузера без втрати кошика та поточного статусу замовлення.

4.2.4 Вимоги до сумісності

Підтримуються браузери Chrome 90+, Safari 14+, Firefox 88+. Підтримувані мобільні платформи: iOS 13+, Android 8+. Сканування QR-коду здійснюється стандартною камерою без встановлення додаткових застосунків.

4.2.5 Вимоги до логування

Усі HTTP-запити та WebSocket-події логуються у структурованому JSON-форматі з обов'язковими полями: `timestamp`, `request_id` для наскрізного трасування, тип події, статус-код. Фінансові операції записуються в окремий незмінний `audit log` з мінімальним терміном зберігання 3 роки. Особисті дані (паролі, токени, платіжні дані) не потрапляють до логів.

4.2.6 Вимоги до моніторингу

Система надає `health-check` ендпоінт із перевіркою стану всіх критичних залежностей (база даних, платіжний шлюз, OAuth-сервіс). Метрики збираються в реальному часі: кількість активних з'єднань, час відповіді, частота помилок. При перевищенні порогів автоматично формуються алерти.

4.3 Use Case діаграми системи за ролями

4.3.1 Use Case діаграма ролі «Гість»

На рисунку 4.1 зображено use case діаграму для ролі «Гість», що охоплює два режими користування: анонімний (доступ через сканування QR-коду без реєстрації) та авторизований (з активним обліковим записом, створеним через email-реєстрацію або Google OAuth). Анонімний гість має доступ до базового циклу замовлення — сканування QR, перегляд меню з кастомізацією страв, формування кошика з групами подачі, відстеження статусу приготування, виклик офіціанта та оплата. Авторизований гість додатково отримує доступ до історії замовлень, залишення відгуків та керування власним профілем.

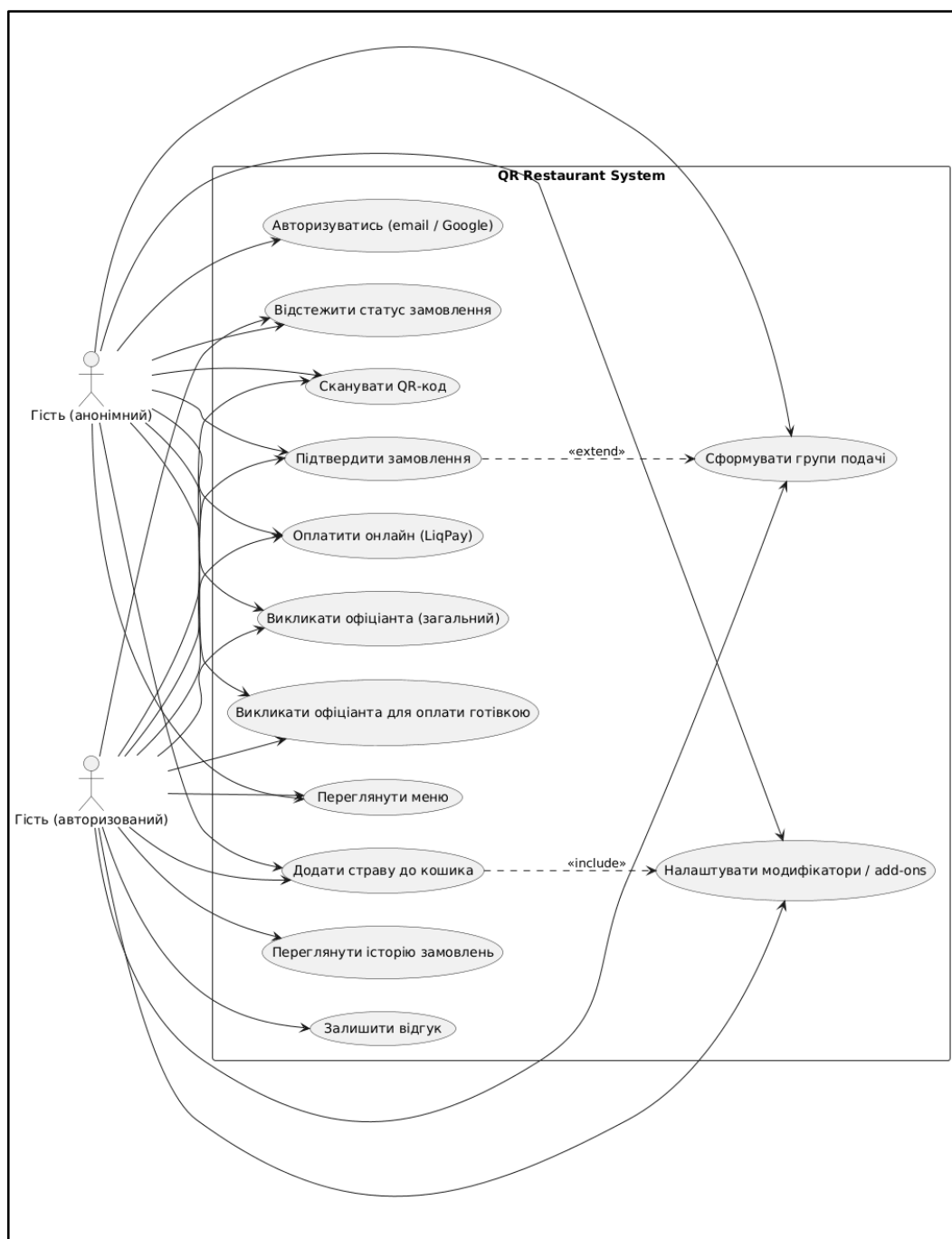


Рисунок 4.1 — Use Case діаграма ролі «Гість» (анонімний та авторизований)

4.3.2 Use Case діаграма ролі «Офіціант»

На рисунку 4.2 зображено use case діаграму для ролі «Офіціант». Основним робочим простором офіціанта є інтерактивна карта залу, де відображається поточний стан кожного столика. Через спеціалізовану панель офіціант підтверджує виклики гостей (загальні та для готівкової оплати), додає позиції до активних

замовлень за зверненням гостя, скасовує замовлення з обов'язковим зазначенням причини, закриває столики вручну та вмикає режим аварійного відновлення сесії гостя.

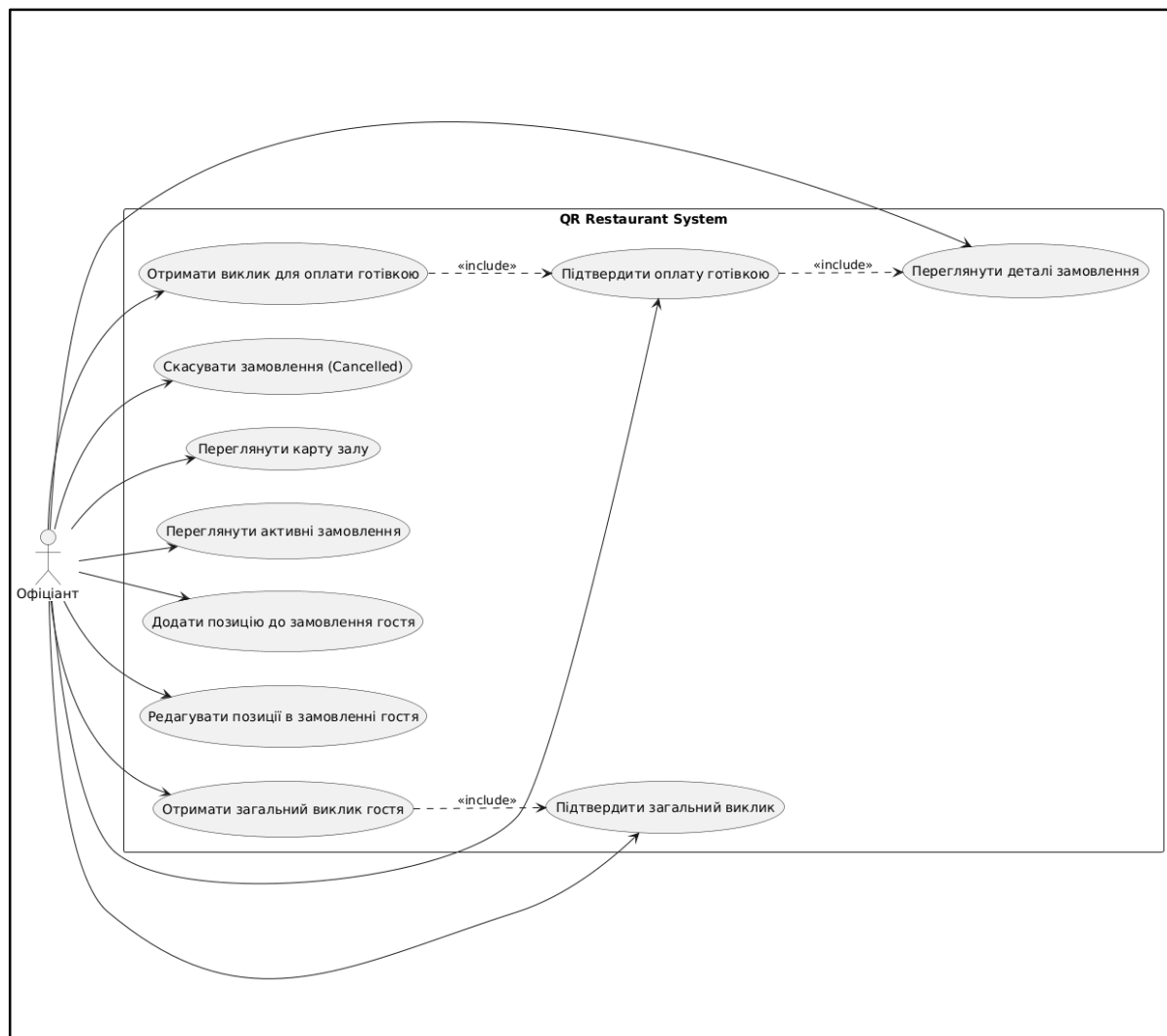


Рисунок 4.2 — Use Case діаграма ролі «Офіціант»

4.3.3 Use Case діаграма ролі «Кухар»

На рисунку 4.3 зображено use case діаграму для ролі «Кухар». Кухар взаємодіє з системою виключно через інтерфейс кухонного дисплея (KDS), де отримує нові замовлення в режимі реального часу через WebSocket. Сфера відповідальності кухаря охоплює керування статусами груп подачі за моделлю waiting → cooking → ready → served, перемикавання між режимами відображення

Order View та Table View, а також управління стоп-листом — додавання та зняття позицій із меню, інгредієнтів, додатків.

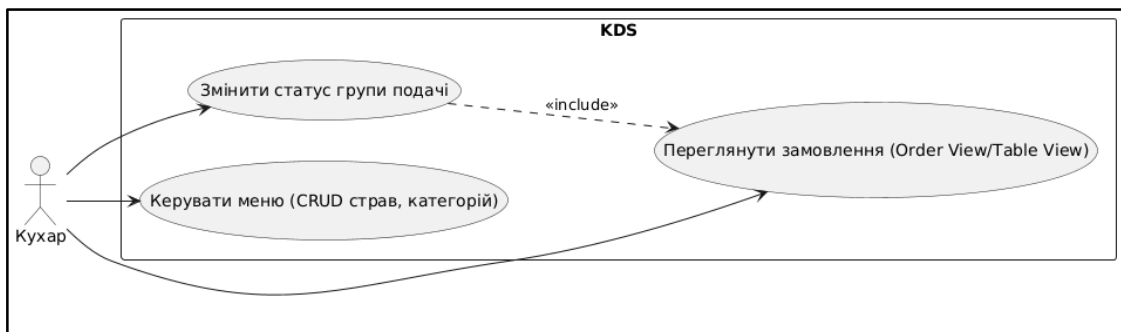


Рисунок 4.3 — Use Case діаграма ролі «Кухар»

4.3.4 Use Case діаграма ролі «Адміністратор»

На рисунку 4.4 зображено use case діаграму для ролі «Адміністратор». Адміністратор має повний доступ до всіх модулів системи: управління меню (CRUD для категорій, страв, інгредієнтів, додатків та груп компонентів), управління персоналом (створення акаунтів, призначення ролей, скидання паролів, деактивація), управління столиками та QR-кодами, перегляд аналітичного дашборду, генерація PDF-меню, модерація відгуків та налаштування параметрів ресторану. Окрім адміністративних функцій, адміністратор також має доступ до операційних інтерфейсів офіціанта та кухаря.



Рисунок 4.4 — Use Case діаграма ролі «Адміністратор»

4.4 Вимоги до тарифної моделі

Система реалізує модель Freemium з технічним розподілом функціоналу на рівні бази даних та middleware. Кожен ресторан характеризується значенням поля `plan` $\in \{\text{free}, \text{premium}\}$, яке визначає набір доступних функцій. Детальний опис розподілу функціоналу між тарифами наведено у п. 3.5 цього звіту.

З точки зору формування вимог тарифний розподіл породжує такі додаткові функціональні вимоги:

- REQ-PLAN-01: Серверна частина перевіряє значення `restaurant.plan` перед обробкою запитів до тарифно-обмежених ендпоінтів (`/payments/initiate`, `/admin/analytics/*`, `/admin/menu/pdf`, `/admin/reviews`, `/admin/staff` для додавання понад 3 акаунтів). При недостатньому рівні підписки повертається HTTP 403 з кодом помилки `PLAN_REQUIRED`.
- REQ-PLAN-02: Серверна частина перевіряє кількісні обмеження для тарифу `free` при операціях створення сутностей: максимум 5 категорій меню, 50 страв, 3 зображення на страву, 3 акаунти персоналу сумарно. При перевищенні ліміту повертається HTTP 403 з кодом `PLAN_LIMIT_EXCEEDED`.
- REQ-PLAN-03: Клієнтська частина отримує значення `plan` у складі профілю ресторану та умовно рендерить елементи інтерфейсу: для тарифу `free` приховуються пункти меню «Аналітика», «Управління відгуками», «Генератор PDF», «Інтеграція платежів», а на формах редагування меню відображається залишок ліміту.
- REQ-PLAN-04: При спробі гостя залишити відгук для замовлення в ресторані з тарифом `free` система не відображає форму відгуку та не активує відповідну кнопку.
- REQ-PLAN-05: Інтеграція з Google Translate (для автоматичного перекладу полів вводу) активується лише для тарифу `premium`.

5 АРХІТЕКТУРА ТА ПРОЕКТУВАННЯ

5.1 Загальна архітектура системи

Програмна система реалізована за класичною трирівневою архітектурою (three-tier architecture) клієнт-сервер з розділенням на рівень представлення, рівень бізнес-логіки та рівень зберігання даних. Така архітектура є усталеним рішенням для веб-застосунків і забезпечує чітке розділення відповідальностей між компонентами, що, у свою чергу, підвищує супроводжуваність, тестованість та масштабованість системи.

На рисунку 5.1 зображено загальну архітектурну схему системи з усіма основними компонентами та зовнішніми залежностями.

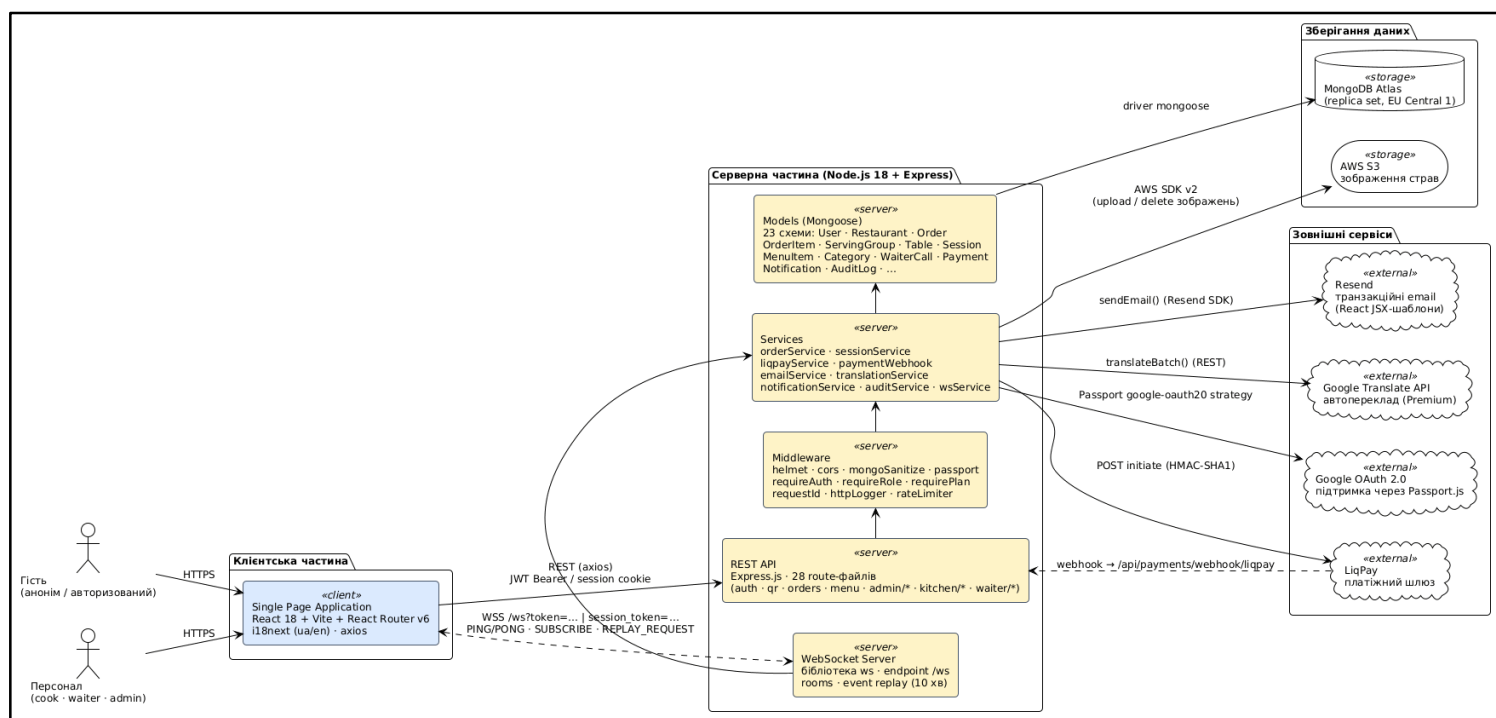


Рисунок 5.1 — Загальна архітектурна схема системи QR Restaurant

Рівень представлення реалізований у вигляді односторінкового застосунку (Single Page Application, SPA) на основі бібліотеки ReactJS. Один і той самий кодовий базис обслуговує три типи користувацьких інтерфейсів — мобільний інтерфейс гостя (із підтримкою Progressive Web App), панель персоналу (офіціант,

кухар) та адміністративну панель — з умовним рендерингом залежно від ролі автентифікованого користувача. Розгортання клієнтської частини здійснюється у вигляді статичних артефактів (HTML, JavaScript, CSS), що віддаються через CDN.

Рівень бізнес-логіки реалізований як серверний застосунок на платформі Node.js 18 LTS із використанням фреймворку Express.js. Сервер виконує дві основні функції: обробляє REST API-запити від клієнтських застосунків та підтримує WebSocket-з'єднання для двосторонньої передачі подій у режимі реального часу. Усі вхідні запити проходять конвеєр проміжного програмного забезпечення (middleware): автентифікація, авторизація (RBAC), перевірка тарифного плану, валідація вхідних даних, обробка помилок.

Рівень зберігання даних реалізований на основі документно-орієнтованої бази даних MongoDB 6.0+. Обґрунтування вибору NoSQL-рішення детально розглянуто у п. 5.4. Файлові ресурси (зображення страв) зберігаються в об'єктному сховищі AWS S3 з окремим bucket на кожен ресторан.

Зовнішніми сервісами системи є: LiqPay (платіжний шлюз для онлайн-оплати), Google OAuth 2.0 (автентифікація через Google-акаунт), Resend (доставка транзакційних email-повідомлень), Google Translate API (опціональний переклад двомовних полів для тарифу Premium).

Архітектурний стиль системи можна охарактеризувати як multi-tenant SaaS (програмне забезпечення як послуга з підтримкою множинної оренди): одна інстанція системи обслуговує багатьох клієнтів-ресторанів з повною ізоляцією даних на рівні префіксу restaurantId у всіх API-маршрутах та індексованих полів у документах MongoDB. Така архітектура є економічно ефективною з точки зору споживання ресурсів та спрощує процес розгортання оновлень одночасно для всіх клієнтів.

5.2 Архітектура клієнтської частини

5.2.1 Технологічний стек та інструменти збірки

Клієнтська частина побудована на базі бібліотеки ReactJS версії 18 з використанням функціональних компонентів та хуків (hooks). Збірка та локальна розробка виконується інструментом Vite — швидким збиральником на основі ES-модулів, що забезпечує миттєве оновлення модулів під час розробки (Hot Module Replacement) та оптимізовану продакшен-збірку з tree-shaking та code-splitting.

Маршрутизація реалізована через React Router v6 з підтримкою захищених маршрутів (Protected Routes), які перевіряють роль автентифікованого користувача перед монтуванням сторінки. Інтернаціоналізація забезпечується бібліотекою i18next з ресурсами української та англійської мов, організованими за принципом «один JSON-файл на сторінку».

5.2.2 Структурна організація компонентів

Структуру клієнтського застосунку зображено на рисунку 5.2.

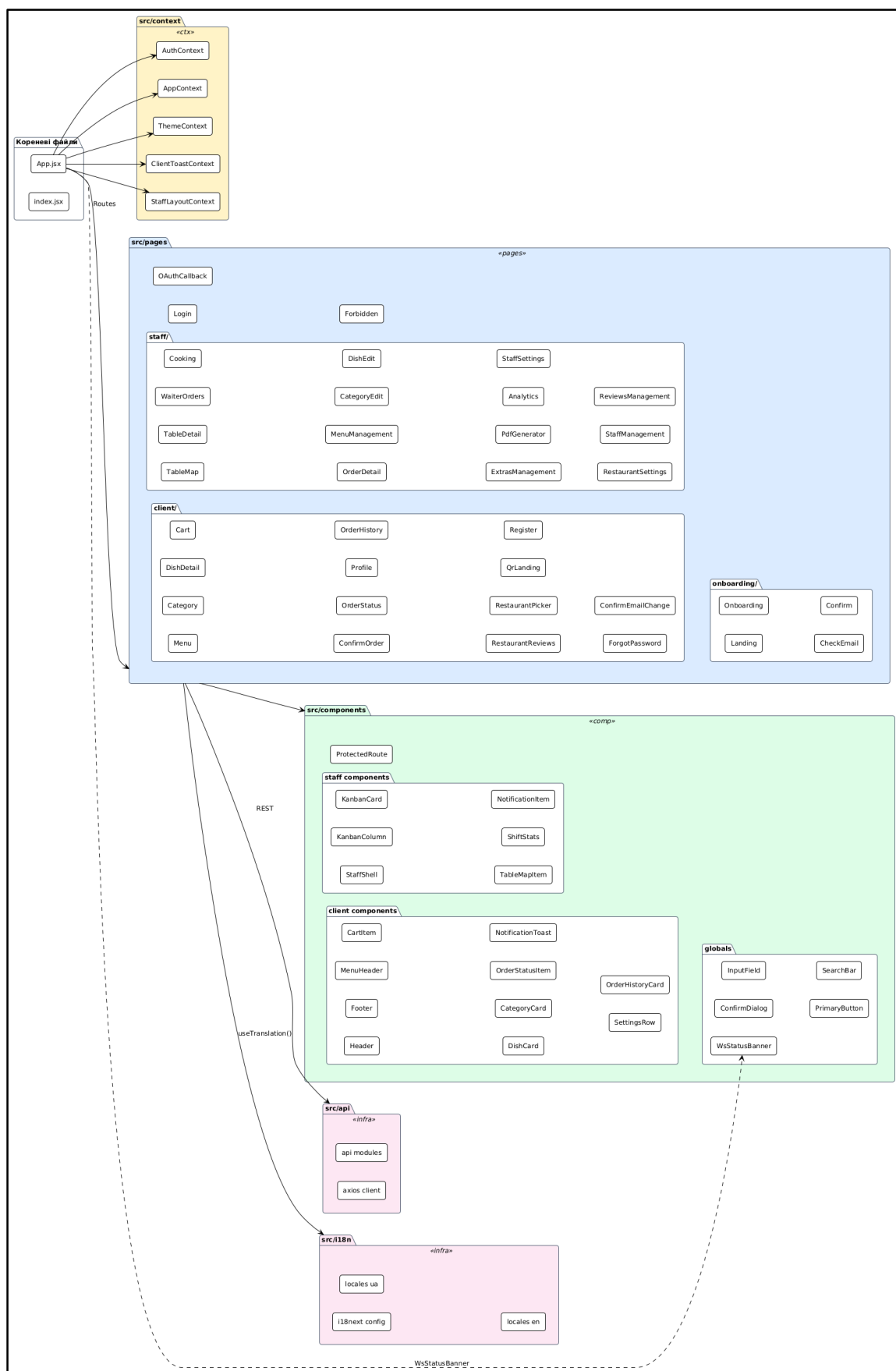


Рисунок 5.2 — Структура клієнтської частини: ієрархія сторінок та контекстів

Каталог `src/pages` містить кореневі сторінки, згруповані за призначенням: `pages/client` (гостьовий інтерфейс), `pages/staff` (панель персоналу), `pages/onboarding` (реєстрація нового ресторану), `pages/Login`. Кожна сторінка є самостійним React-компонентом, обгорнутим у `ProtectedRoute` з зазначенням переліку дозволених ролей.

Каталог `src/components` містить багаторазові компоненти інтерфейсу, серед яких базові примітиви (`InputField`, `Dropdown`, `PrimaryButton`), складені UI-блоки (`StaffShell` — каркас сторінок персоналу з бічним меню), а також спеціалізовані компоненти для real-time повідомлень (`NotificationToast`, `HttpErrorToast`, `WsStatusChip`).

5.2.3 Управління станом через контексти React

Стан застосунку інкапсульований у чотирьох глобальних контекстах React.

`AuthContext` інкапсулює стан автентифікації: профіль користувача, JWT-токени, методи `login`, `logout`, `updateProfile`, `requestEmailChange`, `deleteAccount`. Контекст автоматично оновлює токени за допомогою `refresh`-механізму та опрацьовує сценарій закінчення терміну дії `access`-токена.

`AppContext` інкапсулює стан гостьової сесії: токен сесії, ідентифікатор столика та ресторану, активне замовлення, єдине `WebSocket`-з'єднання, чергу непрочитаних сповіщень. Цей контекст є центральною точкою для real-time оновлень: при отриманні `WebSocket`-події він оновлює внутрішній стан, що автоматично призводить до перерендерингу всіх компонентів-споживачів.

`ThemeContext` керує темою інтерфейсу (світла / темна) з `persistance` у `localStorage` та підтримкою системних налаштувань (`prefers-color-scheme`).

`ClientToastContext` надає глобальний механізм відображення `toast`-повідомлень із чергою та автоматичним прихованням.

5.2.4 Архітектура WebSocket-клієнта

Підключення до WebSocket-сервера інкапсульоване в межах `AppContext` як `singleton` на час життя сесії. Один `WebSocket` використовується всіма компонентами, що передплачують на події (через хук `useWebSocket`), що дозволяє уникнути дублювання з'єднань та зменшує навантаження на сервер. Реалізовано автоматичне відновлення з'єднання за схемою експоненційного відступу (`exponential backoff`) та механізм `event replay`: при відновленні з'єднання клієнт надсилає `last_event_id` і отримує всі пропущені події.

5.2.5 Підтримка PWA та офлайн-режиму

Клієнтський застосунок зареєстрований як `Progressive Web App` через файл `manifest.json` та `Service Worker`. `Service Worker` реалізує дві стратегії кешування: `cache-first` для статичних ресурсів (JS, CSS, шрифти, іконки) та `network-first` з `fallback` для даних меню. Кошик гостя зберігається в `localStorage` та відновлюється при повторному відкритті браузера. Замовлення, надіслане в офлайн-режимі, ставиться в чергу та автоматично доставляється на сервер після відновлення мережі.

5.3 Архітектура серверної частини

5.3.1 Технологічний стек та модульна організація

Серверна частина побудована на платформі `Node.js 18 LTS` з фреймворком `Express.js` для обробки HTTP-запитів та бібліотекою `ws` для підтримки `WebSocket`-з'єднань. Для роботи з `MongoDB` використовується ODM-бібліотека `Mongoose`, що надає рівень моделей даних із валідацією схем та автоматичним керуванням зв'язками між колекціями.

Серверний код організований за принципом шарової архітектури з чітким розділенням відповідальностей. Структура каталогів та потік обробки запиту зображено на рисунку 5.3.

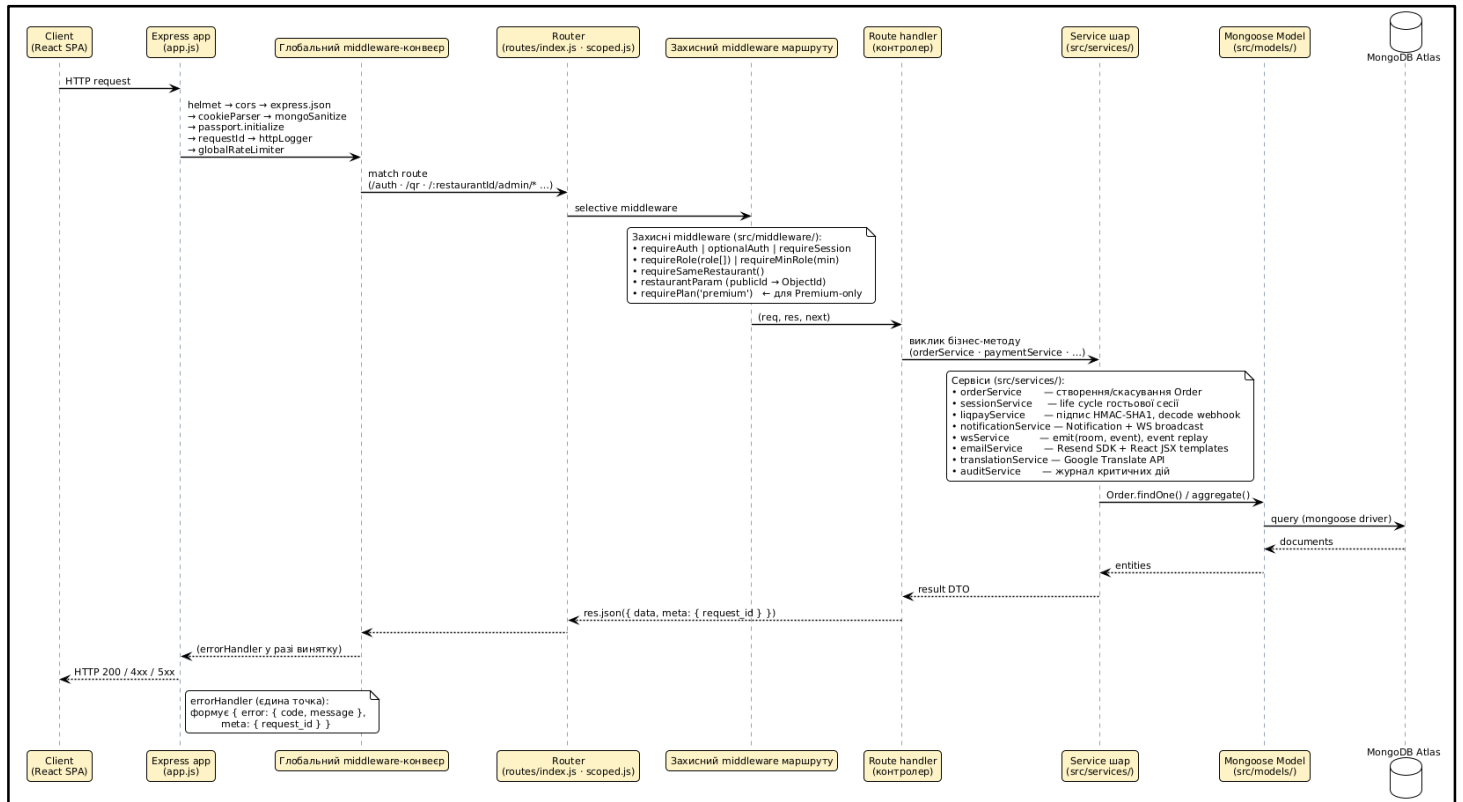


Рисунок 5.3 — Шарова архітектура серверної частини та потік обробки HTTP-запиту

Шарова архітектура серверу включає такі рівні:

— Routes (маршрути) — каталог `src/routes`. Тонкий шар, що відповідає за оголошення URL-маршрутів та делегування обробки до контролерів. Згрупований за бізнес-доменами: `auth.js`, `orders.js`, `menu.js`, `payments.js`, `waiterCalls.js`, `analytics.js` тощо.

— Middleware (проміжне програмне забезпечення) — каталог `src/middleware`. Містить функції, що виконуються до обробника маршруту: `requireAuth` (перевірка JWT-токена), `requireRole(['admin'])` (перевірка ролі), `requirePlan('premium')` (перевірка тарифного плану), `validateBody(schema)` (валідація вхідних даних за схемою `Zod`), `errorHandler` (централізована обробка помилок).

— Services (бізнес-логіка) — каталог `src/services`. Інкапслюють бізнес-правила, незалежні від HTTP-протоколу. Ключові сервіси: `orderService` (створення, оновлення, скасування замовлень, перерахунок статусу столика), `notificationService` (формування та доставка сповіщень персоналу), `paymentWebhookService` (обробка callback від `LiqPay`), `wsService` (broadcast WebSocket-подій до відповідних кімнат).

— Models (моделі даних) — каталог `src/models`. Mongoose-схеми колекцій MongoDB з валідацією, індексами та віртуальними полями. Деталі моделей розглянуто у п. 5.4.

— Utils (допоміжні функції) — генерація short-code, хешування паролів, формування JWT-токенів, перевірка підпису LiqPay.

5.3.2 REST API та принципи проектування

REST API спроектовано відповідно до принципів RESTful: ресурсо-орієнтовані URL, використання HTTP-методів за призначенням (GET для читання, POST для створення, PUT/PATCH для оновлення, DELETE для видалення), стандартні HTTP-коди стану. Усі ресурси ресторану використовують префікс `/:restaurantId/` для повної ізоляції даних: запит до замовлення одного ресторану не може випадково повернути дані іншого. Повну специфікацію API наведено у розділі 6 SRS (Додаток А) та налічує понад 60 ендпоінтів.

Формат відповідей уніфікований: успішна відповідь містить поля `data` та `meta` (з `request_id` для трасування), помилка — поля `error.code` та `error.message`. Пагіновані відповіді додатково містять об'єкт `pagination` з полями `page`, `limit`, `total`, `pages`.

5.3.3 Middleware-конвеєр обробки запитів

Кожен вхідний HTTP-запит проходить наступний конвеєр (приклад для захищеного ендпоінта аналітики):

- `cors` — налаштування заголовків CORS;
- `bodyParser` — парсинг JSON-тіла запиту;
- `requestId` — генерація унікального ідентифікатора запиту для трасування;
- `requireAuth` — перевірка JWT та витяг профілю користувача;
- `requireRole(['admin'])` — перевірка ролі;
- `requirePlan('premium')` — перевірка тарифу ресторану;
- `validateQuery(analyticsQuerySchema)` — валідація query-параметрів;

- контролер маршруту — виконання бізнес-логіки через виклик сервісу;
- errorHandler — централізована обробка винятків та формування відповіді.

5.3.4 Реалізація розподілу тарифів через middleware

Тарифне розмежування функціоналу інкапсульоване в окреме middleware `requirePlan(plan)`, що перевіряє значення поля `plan` у документі ресторану перед обробкою запиту. Аналогічно реалізовано middleware `checkResourceLimit(resource)`, що перевіряє кількісні обмеження безкоштовного тарифу при операціях створення нових сутностей (категорій, страв, акаунтів персоналу). Такий підхід забезпечує єдину точку контролю тарифних обмежень і дозволяє легко змінювати правила без модифікації кожного окремого контролера.

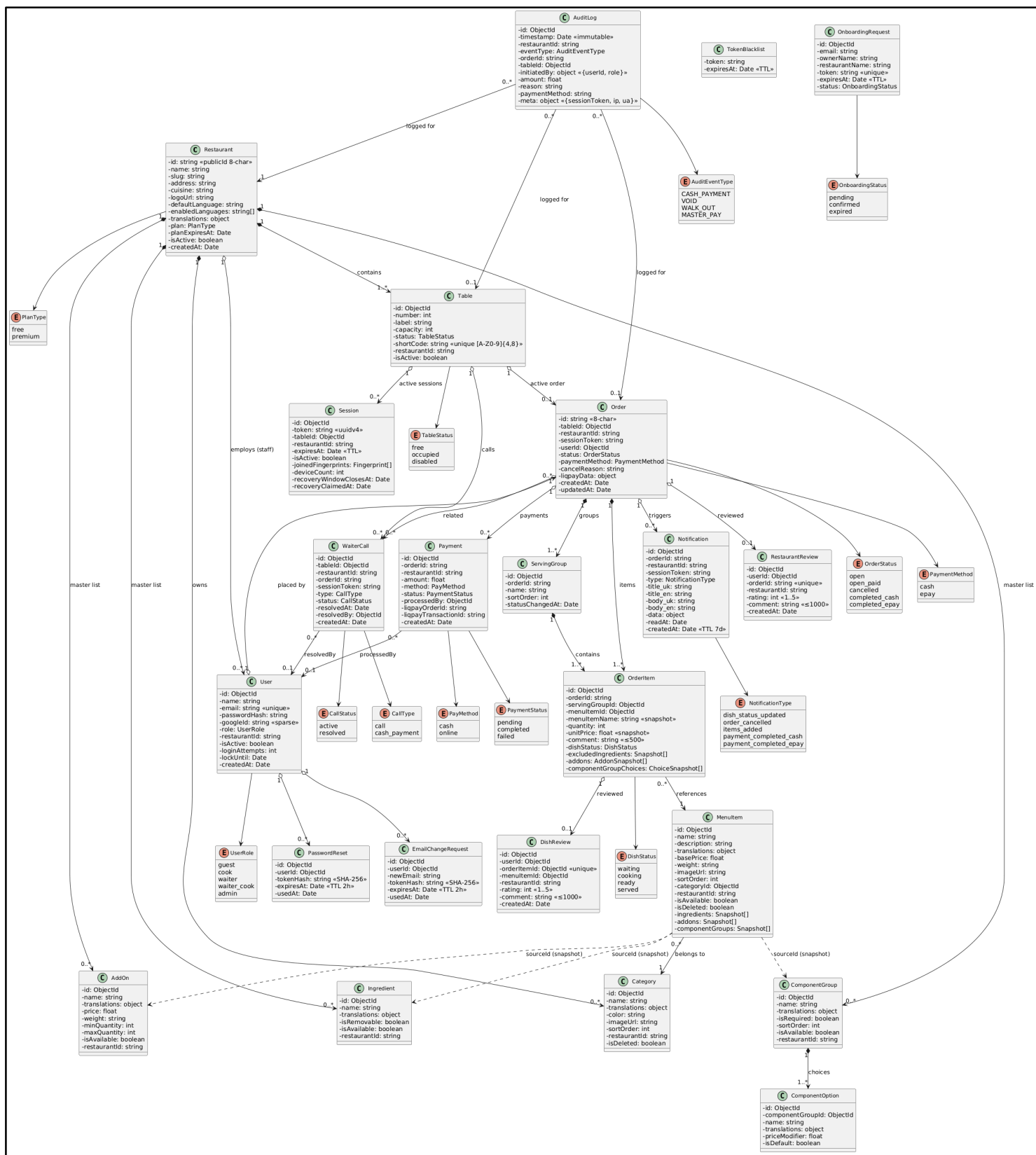
5.4 Проектування бази даних

5.4.1 Обґрунтування вибору MongoDB

Вибір документно-орієнтованої бази даних MongoDB обумовлений природою предметної моделі системи. Меню ресторану має ієрархічну структуру з вкладеними сутностями (страва містить інгредієнти, добавки, групи компонентів), що природно представляється у форматі JSON-документа без необхідності множинних JOIN-операцій. Гнучка схема MongoDB дозволяє оперативно вносити зміни до структури даних без виконання міграцій, що є критичним для проєкту в активній фазі розвитку. Крім того, MongoDB надає вбудовану підтримку TTL-індексів (автоматичне видалення документів за певний час), що використовується для очищення прострочених сесій, токенів скидання пароля та сповіщень.

5.4.2 Огляд ключових колекцій

Domain-модель системи з усіма основними сутностями та зв'язками між ними наведено на рисунку 5.4. Нижче наведено короткий опис ключових колекцій із зазначенням стратегії індексування.



— `restaurants` — кореневі документи ресторанів. Містить поля `name`, `address`, `plan`, `enabledLanguages`, `settings`. Поле `plan` є ключовим для системи тарифного розподілу функціоналу.

— `tables` — столики ресторану. Унікальний індекс на `shortCode` забезпечує миттєвий пошук при скануванні QR. Складений індекс на `(restaurantId, status)` оптимізує запити карти залу для офіціанта.

— `sessions` — гостьові сесії, прив'язані до конкретного столика. TTL-індекс на полі `expiresAt` (2 години) автоматично видаляє прострочені сесії.

— `orders` — замовлення гостей. Складений індекс на `(tableId, status)` критичний для перевірки обмеження одного активного замовлення на столик.

— `orderItems` та `servingGroups` — позиції замовлення та групи подачі.

— `menuItems`, `categories`, `ingredients`, `addOns`, `componentGroups` — сутності меню з підтримкою `soft-delete` через поле `isDeleted`.

— `waiterCalls` — виклики офіціанта з полями `type` (`call` або `cash_payment`) та `status` (`active` або `resolved`).

— `payments` — записи фінансових транзакцій із посиланням на замовлення та методом оплати.

— `reviews` — відгуки гостей (`RestaurantReview` та `DishReview` як єдиний поліморфний документ із полем `type`).

— `notifications` — сповіщення персоналу з TTL-індексом на 7 діб.

— `auditLog` — незмінний журнал критичних операцій (скасування замовлень, готівкові оплати, `walk-out`) з мінімальним терміном зберігання 3 роки.

— `users` — єдина колекція для всіх ролей із полем `role` та опціональним `googleId` (`sparse`-індекс).

5.4.3 Стратегія цілісності даних

Оскільки MongoDB не підтримує `foreign key constraints`, цілісність зв'язків між сутностями забезпечується на рівні застосунку через Mongoose-проміжне програмне забезпечення (`pre/post hooks`) та явні перевірки в сервісному шарі.

Атомарність складних операцій (наприклад, створення замовлення з оновленням статусу столика) реалізована через MongoDB-транзакції на репліка-сеті.

5.4.4 Діаграма стану замовлення

Сутність Order має складний життєвий цикл із чітко визначеним набором станів та однонаправлених переходів між ними. На рисунку 5.5 зображено діаграму стану замовлення, яка відображає всі можливі статуси та умови їх зміни.

Початковим станом замовлення є `open` — замовлення підтверджено гостем та передано на кухню; цей статус не залежить від стадій приготування окремих страв і зберігається до явної фінансово-операційної дії. Зі статусу `open` можливі три переходи: до `open_paid` (при підтвердженні готівкової або онлайн-оплати до закриття столика), до `completed_cash` або `completed_epay` (при закритті замовлення офіціантом разом із підтвердженням типу оплати) та до `cancelled` (скасування замовлення офіціантом або адміністратором з обов'язковим зазначенням причини). Стан `open_paid` є проміжним: він фіксує факт здійсненої оплати, але дозволяє продовжити обслуговування столика; із нього замовлення переходить до фінального стану `completed_*` при закритті столика. Стани `cancelled`, `completed_cash` та `completed_epay` є термінальними — після переходу до них будь-які зміни замовлення заблоковано (HTTP 409).

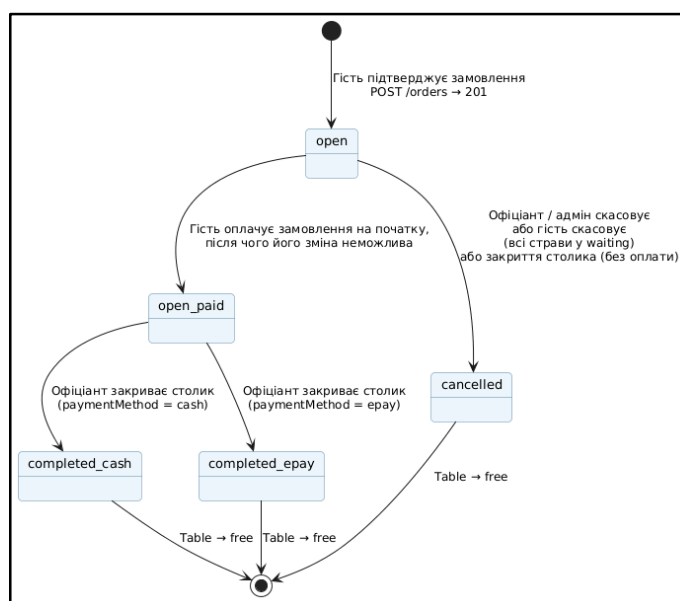


Рисунок 5.5 — Діаграма стану замовлення (Order State Diagram)

5.5 Архітектура взаємодії в реальному часі

5.5.1 Модель кімнат та підписок

Передача подій у реальному часі реалізована через WebSocket-протокол з організацією учасників за моделлю кімнат (rooms). Кімната — це логічна група з'єднань, що отримують одні й ті самі події. Система підтримує чотири типи кімнат:

- `restaurant:{restaurantId}` — широкомовні події для всього персоналу ресторану (зміни меню);

- `table:{tableId}` — події конкретного столика (зміни статусу, виклики офіціанта);

- `session:{sessionToken}` — персональні події гостя (статус власного замовлення, підтвердження оплати);

- `kitchen: {restaurantId}` та `waiter: {restaurantId}` — рольові кімнати персоналу.

При встановленні WebSocket-з'єднання сервер автоматично приєднує учасника до відповідних кімнат на основі типу автентифікації: гість потрапляє до `session:{token}` та `table:{tableId}`, кухар — до `kitchen:{restaurantId}`, офіціант — до `waiter:{restaurantId}`. Архітектуру кімнат зображено на рисунку 5.4.

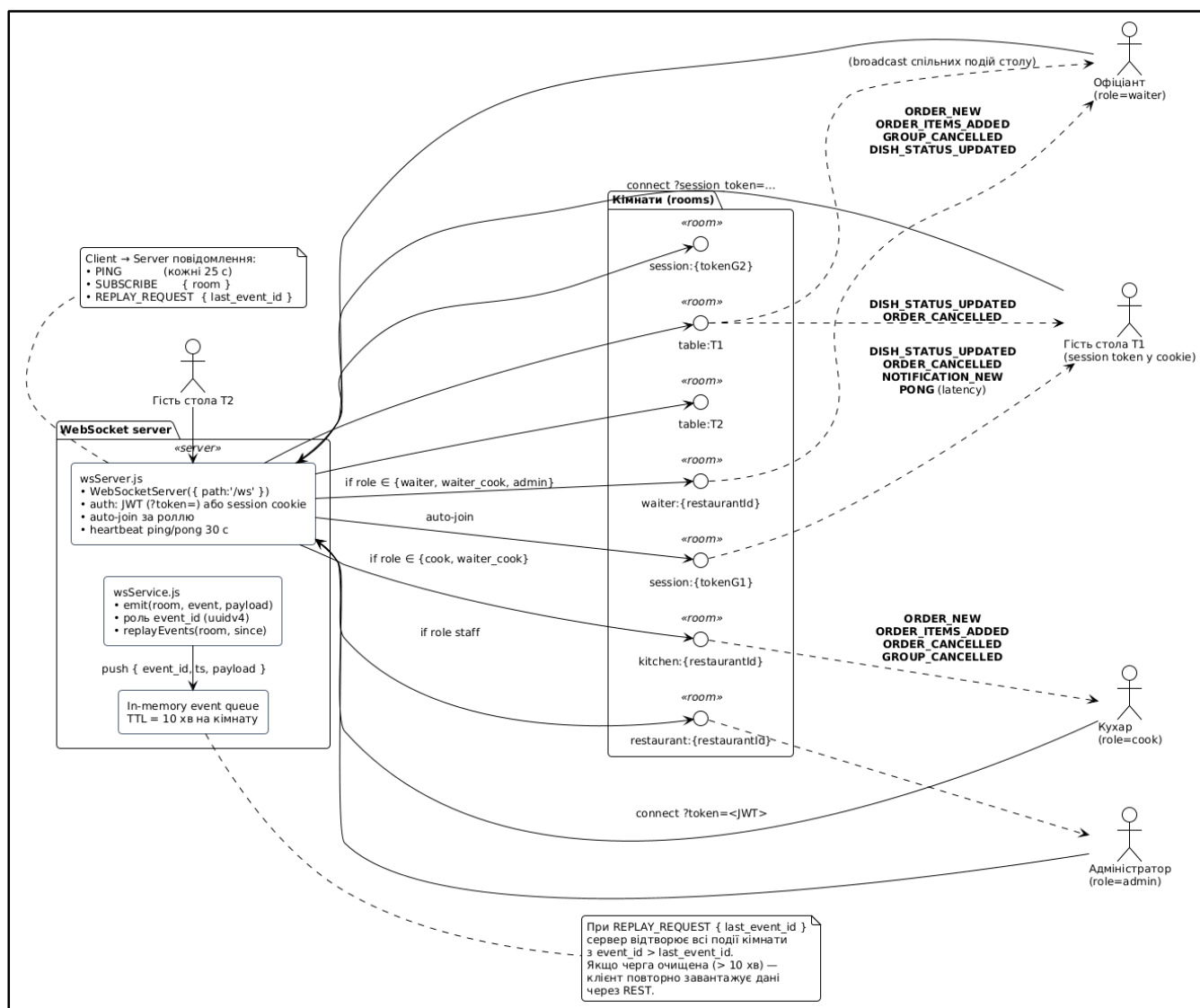


Рисунок 5.5 — Архітектура WebSocket-кімнат та потоки подій

5.5.2 Каталог подій та формат повідомлень

Усі повідомлення мають уніфікований формат із обов'язковими полями event, payload, timestamp та event_id. Поле event_id є унікальним ідентифікатором події, що використовується для механізму відтворення. Повний каталог подій наведено у п. 7.3 SRS і налічує 14 типів серверних подій (наприклад, ORDER_NEW, DISH_STATUS_UPDATED, WAITER_CALL, PAYMENT_COMPLETED).

5.5.3 Механізм event replay та відновлення з'єднання

При розриві WebSocket-з'єднання клієнт автоматично виконує повторні спроби підключення з експоненційними паузами (3 секунди, до 5 спроб). При успішному відновленні з'єднання клієнт надсилає повідомлення `REPLAY_REQUEST` із значенням `last_event_id` — ідентифікатором останньої успішно обробленої події. Сервер зберігає чергу подій тривалістю 10 хвилин на кожну кімнату і відтворює всі події, що надійшли після зазначеного `event_id`. Якщо розрив тривав довше 10 хвилин, клієнт виконує повне перезавантаження даних через REST API.

Поточний стан з'єднання відображається у компоненті `WsStatusChip` у вигляді кольорового індикатора (`Online` / `Reconnecting` / `Offline`), що підвищує прозорість системи для користувача.

5.6 Інтеграція з зовнішніми сервісами

5.6.1 Інтеграція з платіжним шлюзом LiqPay

LiqPay використовується для онлайн-оплати банківськими картками. Інтеграція реалізована за стандартною схемою з ініціацією оплати на клієнтській стороні та підтвердженням транзакції через `webhook` на серверній стороні. Послідовність взаємодії: гість натискає кнопку «Оплатити онлайн» — клієнт надсилає `POST /:restaurantId/payments/initiate { orderId }`; сервер створює запис `Payment` зі статусом `pending`, формує `payload` із даними замовлення та підписує його `HMAC-SHA1` з використанням секретного ключа LiqPay; клієнт відкриває форму LiqPay із підписаним `payload`; гість завершує оплату на стороні LiqPay; LiqPay надсилає асинхронний `webhook` на `POST /api/payments/webhook/liqpay` з результатом транзакції; сервер перевіряє підпис `webhook`, оновлює статус `Payment.status` та переводить замовлення у статус `open_paid` з фіксацією методу оплати `erau`; сервер генерує WebSocket-подію `PAYMENT_COMPLETED` для гостя та персоналу.

Реалізовано fallback-механізм для випадку ненадходження webhook: серверне завдання за розкладом виконує повторні запити статусу до LiqPay API через 1, 5 та 15 хвилин після ініціації. Інтеграція доступна виключно для ресторанів із тарифом Premium.

5.6.2 Інтеграція з Google OAuth 2.0

Google OAuth 2.0 надає альтернативний метод автентифікації для гостей та персоналу. Інтеграція реалізована через бібліотеку Passport.js із стратегією passport-google-oauth20. Користувач натискає «Увійти через Google» — клієнт перенаправляє на GET /auth/google; сервер ініціює OAuth-flow та перенаправляє на сторінку Google; після підтвердження користувачем Google перенаправляє на GET /auth/google/callback?code=...; сервер обмінює code на access-токен Google, отримує профіль користувача (email, ім'я, googleId); сервер перевіряє наявність акаунту з відповідним googleId або email: якщо акаунт існує — виконує вхід, якщо ні — створює новий акаунт; сервер видає JWT access- та refresh-токени та повертає клієнту. Підтримується flow прив'язування Google до існуючого email-акаунта та відв'язування за умови наявності альтернативного методу входу.

5.6.3 Інтеграція з AWS S3 для зберігання зображень

Зображення страв зберігаються в об'єктному сховищі AWS S3 з окремим bucket на кожен ресторан. Завантаження виконується через серверну частину: клієнт надсилає multipart-форму на POST /:restaurantId/admin/menu/items/{itemId}/image, сервер валідує MIME-тип (JPG, PNG, WebP) та розмір (≤ 5 МБ), після чого виконує upload до S3 та зберігає публічний URL у документі страви. Такий підхід забезпечує контроль над вхідним контентом та можливість попередньої обробки зображень (resize, оптимізація).

5.6.4 Інтеграція з Resend для транзакційних email

Сервіс Resend використовується для доставки трьох типів транзакційних email-повідомлень: підтвердження email при onboarding нового ресторану, скидання пароля, підтвердження зміни email-адреси. Шаблони листів реалізовані як React-компоненти з використанням бібліотеки react-email, що дозволяє підтримувати їх у тому самому форматі, що й основний клієнтський інтерфейс.

5.6.5 Інтеграція з Google Translate API

Для ресторанів із тарифом Premium активується інтеграція з Google Translate API для автоматичного перекладу двомовних полів вводу (назв та описів категорій, страв, інгредієнтів). При редагуванні поля адміністратор може натиснути кнопку «Auto-translate», після чого клієнт надсилає запит на проксі-ендпоінт `POST /:restaurantId/admin/translate` (для приховування API-ключа від клієнта), а сервер виконує запит до Google Translate API і повертає перекладений текст.

5.7 Безпека та автентифікація

5.7.1 Модель автентифікації

Система підтримує два типи автентифікації. JWT-автентифікація для авторизованих користувачів (гостей з акаунтом та персоналу): access-токен має термін дії не більше 1 години, refresh-токен — 30 діб. Токени передаються через заголовок `Authorization: Bearer {token}`. При закінченні access-токена клієнт автоматично виконує refresh через `POST /auth/refresh`.

Session-токен для анонімних гостей: криптографічно стійкий випадковий токен (32 байти, base64-кодування), згенерований при скануванні QR-коду та збережений у `HttpOnly Secure cookie` з атрибутами `SameSite=Strict` та `Max-Age=86400` (24 години).

5.7.2 Авторизація через RBAC

Авторизація реалізована за моделлю Role-Based Access Control. Кожен користувач має одну з ролей: `guest`, `cook`, `waiter`, `waiter_cook`, `admin`. Перевірка ролі виконується `middleware requireRole(...)` для кожного захищеного ендпоінта. Додатково реалізована перевірка приналежності до ресторану: користувач не може виконати операцію над сутностями ресторану, до якого він не належить (HTTP 403).

5.7.3 Захист від типових атак

Реалізовано стандартні механізми захисту:

- XSS: усі вхідні дані екрануються; CSP-заголовки обмежують джерела завантаження ресурсів.
- CSRF: для всіх POST/PUT/PATCH/DELETE запитів використовується `SameSite=Strict` cookie + перевірка `Origin` заголовка.
- NoSQL-ін'єкції: вхідні дані валідуються через схеми `Zod` перед передачею в `Mongoose`-запити; оператори типу `$gt`, `$ne` в полях запиту блокуються.
- Rate limiting: ендпоінт `POST /auth/login` обмежений до 5 спроб за 15 хвилин на IP-адресу для запобігання brute-force атакам.
- Захист webhook: усі webhook від `LiqPay` перевіряються через HMAC-SHA1 підпис з секретним ключем.

5.7.4 Зберігання паролів та секретів

Паролі користувачів зберігаються виключно у вигляді `bcrypt`-хешів з параметром `salt-rounds` ≥ 12 . Токени скидання пароля та підтвердження зміни email зберігаються у вигляді `SHA-256` хешу, що унеможливило використання токена навіть у разі компрометації бази даних. Секретні ключі (`LiqPay secret`, `JWT secret`, `AWS credentials`, `Google Auth secrets`) зберігаються у захищеному сховищі секретів і ніколи не потрапляють до коду чи системи контролю версій.

5.8 Сценарії взаємодії компонентів системи

Для ілюстрації динамічної взаємодії між компонентами системи, описаними у попередніх підрозділах, наведено три sequence-діаграми ключових сценаріїв: повний цикл обслуговування гостя без помилок, обробка помилок платіжного шлюзу та механізм відновлення WebSocket-з'єднання після розриву. Ці сценарії охоплюють як основний потік (happy path), так і критичні випадки відмов, що дозволяє оцінити взаємодію всіх архітектурних шарів у комплексі.

5.8.1 Повний цикл обслуговування гостя

На рисунку 5.6 зображено sequence-діаграму повного циклу взаємодії гостя із системою у нормальному сценарії без помилок: сканування QR-коду → виконання Smart Session Recovery або генерація нової сесії → перегляд меню з гостьовою кастомізацією страв → формування кошика з групами подачі → підтвердження замовлення з передачею на KDS через WebSocket-подію ORDER_NEW → отримання real-time оновлень статусів страв через DISH_STATUS_UPDATED → ініціація онлайн-оплати через LiqPay → отримання webhook-підтвердження та переведення замовлення у статус open_paid. Діаграма демонструє послідовну взаємодію клієнтського SPA, REST API, WebSocket-сервера, бази даних MongoDB та зовнішнього платіжного шлюзу.

5.8.2 Обробка помилок платіжного шлюзу

На рисунку 5.7 зображено sequence-діаграму обробки помилкових сценаріїв при взаємодії з платіжним шлюзом LiqPay. Розглянуто чотири типи відмов: timeout відповіді шлюзу при ініціації платежу, явна відмова шлюзу під час перевірки картки, відхилений платіж (недостатньо коштів, заблокована картка) та ненадходження webhook-підтвердження протягом визначеного часу. Для останнього випадку відображено fallback-механізм: серверне завдання за

розкладом виконує повторні запити статусу транзакції через 1, 5 та 15 хвилин після ініціації, що забезпечує консистентність між станом замовлення в системі та фактичним результатом транзакції на стороні шлюзу.

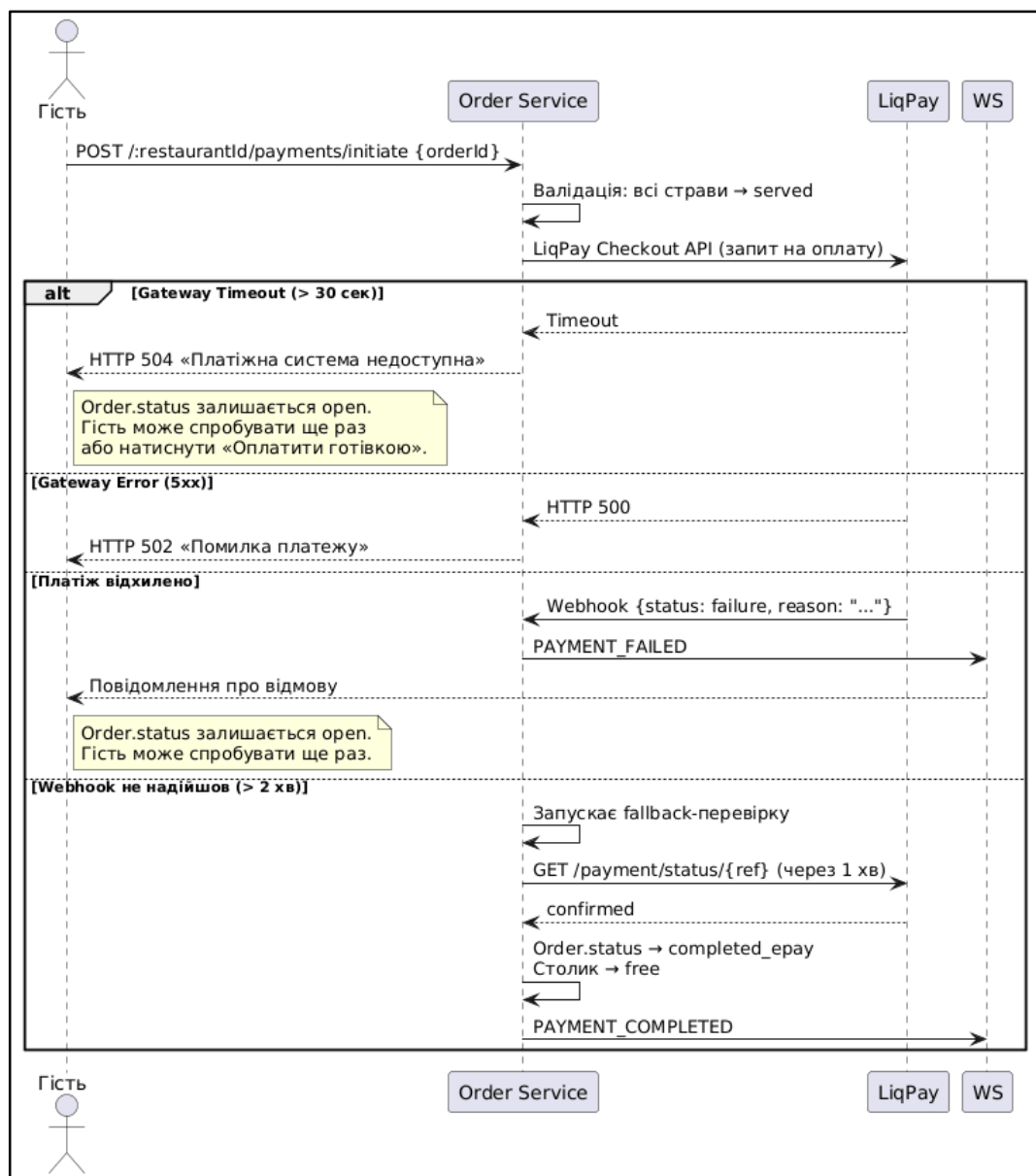


Рисунок 5.7 — Sequence-діаграма обробки помилок платіжного шлюзу та fallback-механізму

5.8.3 Відновлення WebSocket-з'єднання та event replay

На рисунку 5.8 зображено sequence-діаграму механізму відновлення WebSocket-з'єднання після розриву, що є критичним для забезпечення цілісності

real-time-комунікації між гостем, кухнею та офіціантом. Сценарій охоплює: виявлення клієнтом розриву з'єднання, автоматичні повторні спроби підключення з експоненційними паузами, надсилання повідомлення `REPLAY_REQUEST` із значенням останньої успішно обробленої події (`last_event_id`), пошук пропущених подій сервером у in-memory черзі з TTL 10 хвилин та послідовне відтворення пропущених подій клієнту. У випадку, коли розрив тривав довше за термін зберігання черги, клієнт виконує повне перезавантаження стану через REST API.

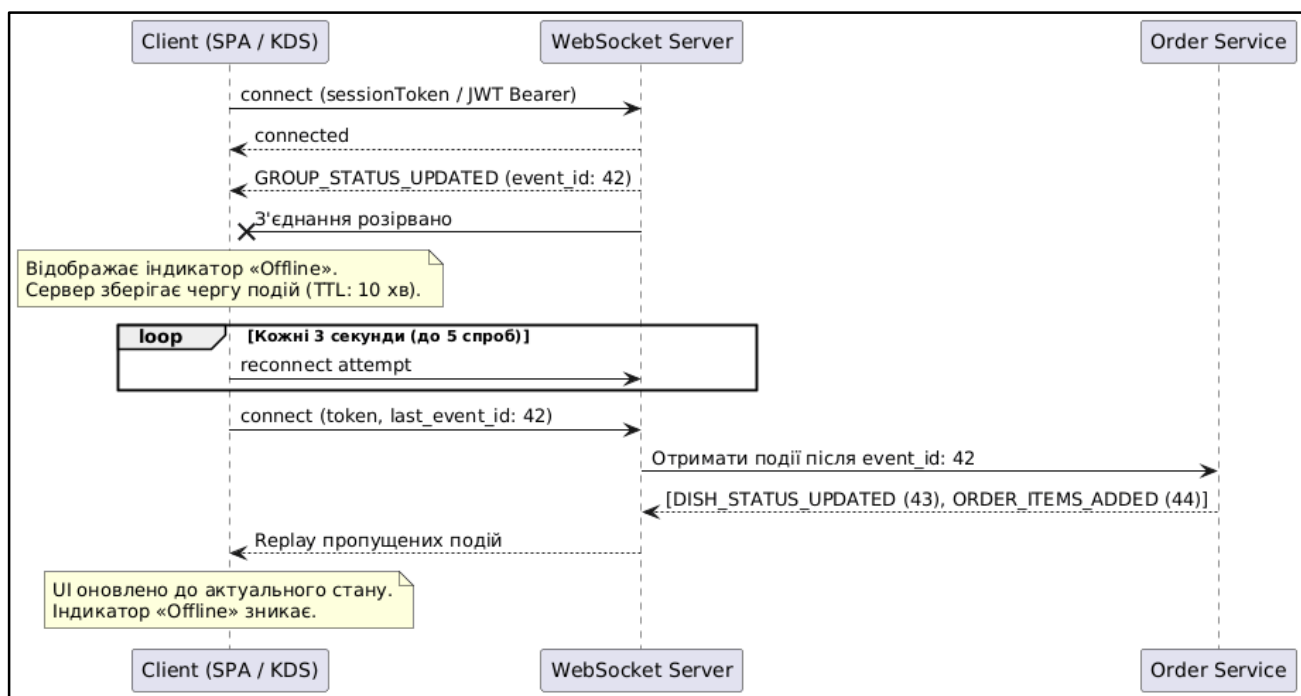


Рисунок 5.8 — Sequence-діаграма відновлення WebSocket-з'єднання та механізму event replay

ВИСНОВКИ

У ході виконання передатестаційної практики спроектовано та реалізовано програмну систему для безконтактного замовлення їжі в ресторані з використанням QR-технологій. Створена система забезпечує повний цикл обслуговування гостя від моменту сканування QR-коду на столику до завершення оплати та залишення відгуку, замінюючи традиційну модель «гість — офіціант — кухня — офіціант — гість».

У межах роботи вирішено такі завдання:

1. Виконано детальний аналіз предметної галузі та порівняльний аналіз чотирьох існуючих програмних рішень (Poster POS, OddMenu, Choice QR, Kavapp QR). За результатами аналізу виявлено ринкову прогалину: існуючі рішення або є фінансово недоступними для закладів малого формату через надлишковий функціонал, або обмежуються виключно функцією відображення меню без можливості повноцінного замовлення гостем.

2. Сформовано повний комплект функціональних та нефункціональних вимог до системи. Сформульовано понад 80 функціональних вимог з унікальними ідентифікаторами, розподілених на 15 модулів, та понад 70 критеріїв приймання у форматі Given/When/Then. Результати оформлено у вигляді документа Software Requirements Specification (SRS) (Додаток А).

3. Спроектовано трирівневу клієнт-серверну архітектуру системи з використанням обґрунтовано підібраного технологічного стеку (ReactJS, Node.js, MongoDB). Розроблено схему бази даних із 20+ колекціями та стратегією індексування для оптимізації типових запитів.

4. Реалізовано клієнтську частину у вигляді односторінкового застосунку з підтримкою Progressive Web App, що забезпечує часткову роботу в офлайн-режимі та зберігає кошик гостя при втраті з'єднання. Інтерфейс адаптований під три типи користувачів — гостя, персонал та адміністратора — з умовним рендерингом залежно від ролі.

5. Реалізовано серверну частину з шаровою архітектурою, що включає понад 60 REST API-ендпоінтів та повноцінну систему передачі подій у реальному часі через WebSocket з моделлю кімнат та механізмом event replay. Розроблено систему автентифікації з підтримкою email+пароль та Google OAuth 2.0 для всіх типів користувачів.

6. Реалізовано розширені модулі системи: аналітичний дашборд, генератор PDF-меню, систему відгуків, інтеграцію з платіжним шлюзом LiqPay для онлайн-оплати та повноцінну систему виклику офіціанта з підтримкою двох типів викликів.

7. Реалізовано модель монетизації Freemium із технічним розподілом функціоналу на рівні бази даних та middleware: безкоштовний тариф включає базовий функціонал прийому та обслуговування замовлень з обмеженнями на обсяг меню та кількість акаунтів персоналу; розширений тариф знімає обмеження та активує модулі онлайн-оплати, аналітики, генератора PDF, системи відгуків та автоматизованого перекладу через Google Translate.

Усі поставлені у вступі завдання виконано в повному обсязі. Створений програмний продукт є придатним для реального впровадження в закладах громадського харчування малого та середнього формату. Модульна архітектура та чітко документована специфікація вимог забезпечують можливість подальшого розширення функціоналу без перепроєктування системи.

Перспективи подальшого розвитку

Архітектура системи спроектована з урахуванням можливості подальшого розширення функціональності. Перспективними напрямками розвитку є такі.

Розширення мовної підтримки. Поточна реалізація підтримує українську та англійську мови. Завдяки використанню бібліотеки `i18next` з ресурсами, винесеними в окремі JSON-файли за принципом «один файл на сторінку», додавання нової мови потребує виключно перекладу ресурсних файлів без

модифікації кодової бази. Перспективними є мови країн Європейського Союзу для виходу системи на міжнародний ринок.

Розширення тем інтерфейсу. Поточна реалізація підтримує світлу та темну теми через систему CSS-змінних. Архітектура передбачає можливість додавання додаткових тем (наприклад, високої контрастності для людей із вадами зору, сезонних оформлень) шляхом додавання нових наборів CSS-змінних.

Розширення функціоналу. Перспективними додатковими модулями є: підтримка доставки за межі закладу з інтеграцією служб доставки, реалізація функції розділення рахунку (Split Bill) між кількома гостями за одним столиком, інтеграція системи бронювання столиків, реалізація програми лояльності з накопичувальними бонусами та промокодами.

Розвиток генератора PDF-меню. Поточна реалізація використовує єдині шаблони оформлення. Перспективним є створення повноцінного редактора з бібліотекою налаштовуваних шаблонів, можливістю редагування розташування елементів, вибором шрифтів, кольорової палітри та фонових зображень, що дозволить адміністраторам формувати фізичні меню без залучення дизайнерів.

Поглиблення аналітичного блоку. Перспективним є розширення аналітики такими напрямками: погодинна та посегментна аналітика навантаження для оптимізації графіків роботи персоналу, прогнозування попиту на основі історичних даних, ABC-аналіз страв за прибутковістю, аналіз ефективності окремих офіціантів за середнім чеком та швидкістю обслуговування, інтеграція даних про погодні умови та локальні події для виявлення зовнішніх факторів впливу на виручку.

Інтеграція зі штучним інтелектом. Найперспективнішим напрямом є впровадження ШІ-функціоналу для різноманітних задач: автоматична генерація описів страв на основі назви та інгредієнтів, персоналізовані рекомендації страв для авторизованих гостей на основі історії замовлень, чат-бот для відповідей на типові запитання гостей (склад страв, алергени, час приготування), автоматичний аналіз тональності відгуків для виявлення проблемних аспектів обслуговування,

прогнозування продажів та автоматичні рекомендації щодо коригування меню та цін на основі попиту.

Інтеграція з POS-системами. Перспективною є двостороння інтеграція з популярними POS-системами (Poster, Syrve, Servio) для синхронізації меню, замовлень та фінансових даних, що дозволить впроваджувати систему в заклади, які вже використовують існуюче програмне забезпечення.

Мобільні застосунки для персоналу. У перспективі можлива розробка нативних мобільних застосунків (iOS, Android) для персоналу як альтернативи веб-інтерфейсу, з можливістю push-сповіщень про нові замовлення та виклики офіціанта на рівні операційної системи.

Реалізація запропонованих напрямів розвитку забезпечить трансформацію системи з вузькоспеціалізованого SaaS-рішення на повноцінну цифрову платформу для управління закладами громадського харчування з підтримкою повного спектру операційних та маркетингових задач.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАНЬ

[1] Звягінцева О. Проблеми ресторанного бізнесу в Україні у 2025 році та рішення для них [Електронний ресурс]. URL: <https://hub.kyivstar.ua/articles/problemi-restorannogo-biznesu-v-ukrayini-u-2025-roczyi-ta-rishennya-dlya-nih> (дата звернення: 16.04.2026).

[2] Г. Островська, Р. Шерстюк, О. Летун. Аналіз ринкових тенденцій інтелектуальної автоматизації ресторанного бізнесу [Електронний ресурс]. URL: [https://doi.org/10.32515/2663-1636.2023.10\(43\).143-155](https://doi.org/10.32515/2663-1636.2023.10(43).143-155) (дата звернення: 16.04.2026).

[3] В. Силівейстр. Топ 13 трендів у ресторанному бізнесі у 2026 році [Електронний ресурс]. URL: <https://joinposter.com/ua/post/restoranni-trendy> (дата звернення: 17.04.2026).

[4] М. Турчиняк, А. Примака. Вплив пандемії covid-19 на готельно-ресторанну індустрію України [Електронний ресурс]. URL: <https://doi.org/10.32782/2524-0072/2023-47-29> (дата звернення: 17.04.2026).

[5] Poster POS: офіційний сайт системи автоматизації [Електронний ресурс]. URL: <https://joinposter.com> (дата звернення: 18.04.2026).

[6] OddMenu: цифрове QR-меню для ресторанів [Електронний ресурс]. URL: <https://oddmenu.com> (дата звернення: 18.04.2026).

[7] Choice QR: система безконтактного замовлення та оплати [Електронний ресурс]. URL: <https://choiceqr.com> (дата звернення: 18.04.2026).

[8] Kavapp: система управління кав'ярнями та ресторанами [Електронний ресурс]. URL: <https://kavapp.com> (дата звернення: 18.04.2026).